8

Reinforcement Learning and AI Agents

In *Chapters 5–7*, we discussed how to provide our model with access to external memory. This memory was stored in a database of a different type (vector or graph), and through a search, we could look up the information needed to answer a question. The model would then receive all the information that was needed in context and then answer, providing definite and discrete real-world information.

However, as we saw later in ***Chapter 7***, LLMs have limited knowledge and understanding of the real world (both when it comes to common-sense reasoning and when it comes to spatial relations).

Humans learn to move in space and interact with the environment through exploration. In a process that is trial and error, we humans learn that we cannot touch fire or how to find our way home. Likewise, we learn how to relate to other human beings through interactions with them. Our interactions with the real world allow us to learn but also to modify our surroundings. The environment provides us with information through perception, information that we process and learn from, and ultimately use to modify the environment. This is a cyclical process in which we sense changes in the environment and respond.

We do not learn all these skills just by reading from a book; it wouldn't be possible. Interaction with the environment, therefore, is critical to learn certain skills and knowledge. Without this, we would find it difficult to do certain tasks. So, we need a system that allows artificial intelligence to interact and learn from the environment through exploration. **reinforcement learning (RL)** is a paradigm that focuses on describing how an intelligent agent can take actions in a dynamic environment. RL governs the behavior of an agent, what actions to take in a given environment (and the state of that environment), and how to learn from it.

In this chapter, therefore, we will discuss RL. We will start with some theory on the topic. We will start with a simple case in which an agent needs to understand how to balance exploration with exploitation in order to find the winning strategy to solve a problem. Once the basics are defined, we will describe how we can use a neural network as an agent. We will look at some of the most popular algorithms used nowadays for interacting and learning from the environment. In addition, we will show how an agent can be used to be able to explore an environment (such as training an agent to solve a video game). In the last section, we will discuss the intersection of **large language models (LLMs)** and RL.

In this chapter, we'll be covering the following topics:

- Introduction to reinforcement learning

- Deep reinforcement learning
- LLM interactions with RL models

Technical requirements

Most of this code can be run on a CPU, but it is preferable to run it on a GPU. This is especially true when we are discussing how to train an agent to learn how to play a video game. The code is written in PyTorch and uses standard libraries for the most part (PyTorch, OpenAI Gym). The code can be found on GitHub: **https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr8**.

Introduction to reinforcement learning

In previous chapters, we discussed a model that learns from a large amount of text. Humans—and increasingly, AI agents—learn best through trial and error. Imagine a child learning to stack blocks or riding a bike. There's no explicit teacher guiding each move; instead, the child learns by acting, observing the results, and adjusting. This interaction with the environment—where actions lead to outcomes and those outcomes shape future behavior—is central to how we learn. Unlike passive learning from books or text, this kind of learning is goal-directed and grounded in experience. To enable machines to learn in a similar way, we need a new approach. This learning paradigm is called RL.

More formally, an infant learns from their interaction with the environment, from the consequential relationship of an action and its effect. A child's learning is not simply exploratory but aimed at a specific goal; they learn what actions must be taken to achieve a goal. Throughout our lives, our learning is often related to our interaction with the environment and how it responds in response to our behavior. These concepts are seen as the basis of both learning theory and intelligence in general.

RL is defined as a branch of machine learning, where a system must make decisions to maximize cumulative rewards in a given situation. Unlike in supervised learning (wherein a model learns from labeled examples) or unsupervised learning (wherein a model learns by detecting patterns in the data), in RL the model learns from experience. In fact, the system is not told what actions it must perform but must explore the environment and find out what actions allow it to have a reward. In more complex situations, these rewards may not be immediate but come only later (e.g., sacrificing a piece in chess but achieving victory later). So, on a general level, we can say that the basics of RL are trial and error and the possibility of delayed reward.

From this, we derive two important concepts that will form the basis of our discussion of RL:

- **Exploration versus exploitation**: The model must exploit previously acquired knowledge to achieve its goal. At the same time, it must explore the environment in order to make better choices in the future. A balance must be struck between these two aspects, because solving a problem may not be the most obvious path. A model therefore must

test different types of actions (explore) before it can exploit the best action (exploitation). Even today, choosing the best balance is an open challenge for RL theory. A helpful way to understand this is by imagining someone trying different restaurants in a new city. At first, they might try a variety of places (exploration) to see what's available. After discovering a few favorites, they might start going to the same ones more often (exploitation). But if they always stick to the familiar spots, they might miss out on finding an even better restaurant. The challenge is knowing when to try something new and when to stick with what works—and this is still an open question in RL.

- **Achieving a global goal in an uncertain environment**: RL focuses on achieving a goal without requiring the problem to be reframed into subproblems. Instead, it addresses a classic supervised machine learning challenge, which involves breaking a complex problem into general subproblems and devising an effective schedule. In the case of RL, on the other hand, one directly defines a general problem that an agent must solve. This does not mean that there has to be only one agent but there can be multiple agents with a clear goal interacting with each other. A relatable example would be learning to commute efficiently in a new city. At first, you don't break the task into subproblems like "learn the bus schedule," "estimate walking time," or "optimize weather exposure." Instead, you treat the goal as a whole: get to work on time every day. Through trial and error—taking different routes, trying trains versus buses, adjusting for traffic—you learn which options work best. Over time, you build a strategy without ever explicitly labeling every part of the problem. If you live with roommates or friends who are doing the same, you might exchange tips or compete for the fastest route, just like multiple agents interacting in RL.

There are several elements that are present in an RL system: an **agent**, the **environment**, a **state**, a **policy**, a **reward signal**, and a **value function**. The agent is clearly the learner or decision-maker (the model that interacts with the environment, makes decisions, and takes actions). The environment, on the other hand, is everything that the environment interacts with. A state represents a particular condition or configuration of the environment at a certain time (for example, the state of the pieces on a chessboard before a move). Given a state, the agent must make choices and choose an action to take. Not all space is always observable; our agent can only have access to a partial description of the state. For example, a robotic agent navigating a maze can only get information through the camera and thus observe only what is in front of it. The information obtained from the camera is an observation, so the model will use only a subset of the state.

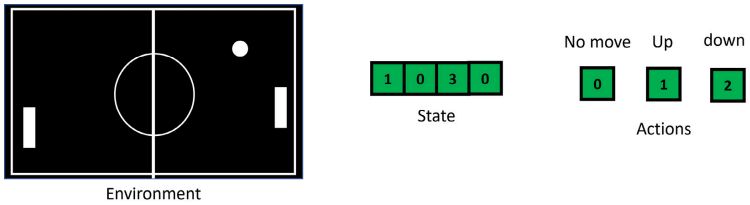


Figure 8.1 – Representation of elements in the RL system

In the preceding figure, we can see how the environment (in this case, the game screen) is represented in vector form (this is the state). Also, the

three possible actions are represented in this case with a scalar. This allows us to be able to train an algorithm.

Actions are the possible decisions or moves that an agent can conduct in an environment (the pieces on the chessboard can only move in certain directions: a bishop only diagonally, the rook vertically or horizontally, and so on). The action set can be discrete (movements in the maze) but also a continuous action space (in this case, it will be real-value vectors). These actions are part of a strategy to achieve a certain goal, according to the state of the environment and policy.

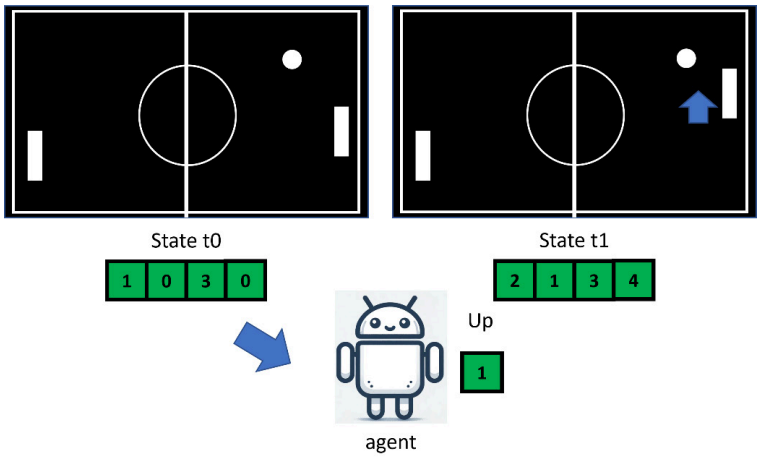


Figure 8.2 – Interaction of the agent with the environment selecting an action

In the preceding figure, we can see that time 0 (*t0*) corresponds to a state *t0*; if our agent acts with a move, this changes the environment. At time *t1*, the environment will be different and therefore we will have a different state, *t1*.

The **policy** defines how an agent behaves at a certain time. Given, then, the state of the environment and the possible actions, the policy maps the action to the state of the system. The policy can be a set of rules, a lookup table, a function, or something else. The policy can also be stochastic by specifying a probability for each action. In a sense, policy is the heart of RL because it determines the agent's behavior. In psychology, this can be defined as a set of stimulus-response rules. For example, a policy might be to eat an opponent's piece whenever the opportunity arises. More often, a policy is parameterized: the output of the policy is a computable function that depends on a set of parameters. One of the most widely used systems is a neural network whose parameters are optimized with an optimization algorithm.

The **reward** is a positive or negative signal received from the environment. It is another critical factor because it provides a goal to the agent at each time step. This reward is used to define both the local and global objectives of an agent. In other words, at each time step, the agent receives a signal from the environment (usually a single number), and in the long run, the agent's goal is to optimize this reward. The reward then allows us to determine whether the model is behaving correctly or not and allows us to understand the difference between positive and negative events, to understand our interaction with the environment and the appropriate response to the state of the system. For example, losing a piece can be considered a local negative reward, and winning the game a global reward.

The reward is often used to change the policy and calibrate it in response to the environment.

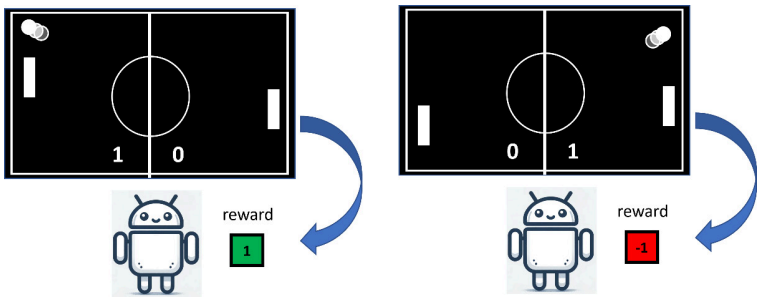


Figure 8.3 – Example of positive and negative reward

The reward, though, gives us information about what is right in the immediate moment while a **value function** defines what is the best approach in the long run. In more technical terms, the value of a state is the total amount of reward that an agent can expect to get in the future, starting from that state (for example, how many points in a game the agent can collect starting from that position). In simple words, the value function helps us understand what happens if we consider that state and subsequent states, and what is likely to happen in the future. There is, however, a dependence between rewards and value; without the former, we cannot calculate the latter, despite that our real goal is value. For example, sacrificing a piece has a low reward but may ultimately be the key to winning the game. Clearly, establishing a reward is much easier, while it is difficult to establish a value function because we have to take into account not only the current state, but all previous observations conducted by the agent.

A classic example of RL is an agent who has to navigate a maze. A state *S* defines the agent's position in the maze; this agent has a possible set of actions *A* (move east, west, north, or south). The policy π indicates what action the agent must take in a certain state. A reward *R* can be a penalty when the agent chooses an action that is not allowed (slamming on a wall, for example), and the value is getting out of the maze. In *Figure 8.4*, we have a depiction of the interactions between an agent and its environment:

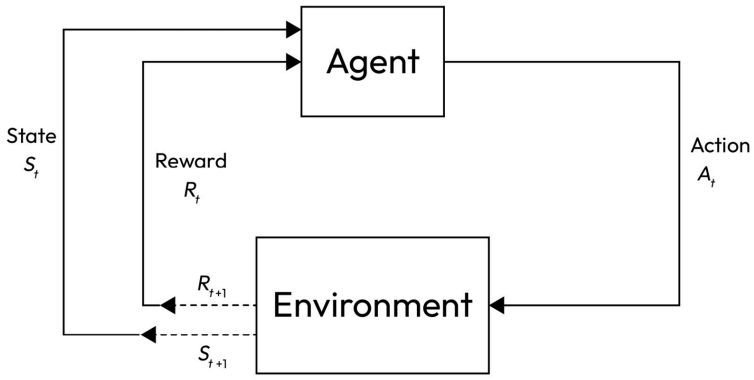


Figure 8.4 – Overview model of reinforcement learning system (**https://arxiv.org/pdf/2408.07712**)

At a given time step (*t*), an agent observes the state of the environment (S_t), chooses an action (A_t) according to policy π, and receives a reward (R_t). At this point, the cycle repeats at the new state (S_{t+1}). The policy can be static or updated at the end of each cycle.

In the next section, we will begin to discuss an initial example of RL, starting with the classic multi-armed bandit.

The multi-armed bandit problem

The **k-armed bandit problem** is perhaps the most classic example to introduce RL. RL is needed for all those problems in which a model must learn from its actions rather than be instructed by a positive example. In the *k*-armed bandit problem, we have a slot machine with *n* independent arms (bandits), and each of these bandits has its own rigged probability distribution of success. Each time we pull an arm, we have a stochastic probability of either receiving a reward or failing. At each action, we have to choose which lever to pull, and the rewards are what we gain. The goal is to maximize our expected total reward over a certain period of time (e.g., 1,000 actions or time steps). In other words, we have to figure out which levers give us the best payoff, and we will maximize our actions on them (i.e., we will pull them more often).

The problem may appear simple, but it is far from trivial. Our agent does not have access to the true bandit probability distribution and must learn the most favorable bandits through trial and error. Moreover, as simple as this problem is, it has similarities to several real-world case scenarios: choosing the best treatment for a patient, A/B testing, social media influence, and so on. At each time step *t*, we can select an A_t action and get the corresponding reward (R_t). The value of an arbitrary action *a*, defined as $q_*(a)$, is the expected reward if we selected this action at time step *t*:

$$q_*(a) = E[R_t \mid A_t = a]$$

If we knew the value of each action, we would have practically solved the problem already (we would always select the one with the highest value). However, we do not know the value of an action, but we can calculate an estimated value, defined as $Q_{t(a)}$, which we wish to be close to $q_*(a)$. At each time step, we have estimated values $Q_{t(a)}$ that are greater than the others; selecting these actions (pulling the arms) is called greedy actions and exploiting the current knowledge. Conversely, selecting an action with a lower estimated value is referred to as exploration (because it allows us to explore what happens with other actions and thus improve our estimation of these actions).

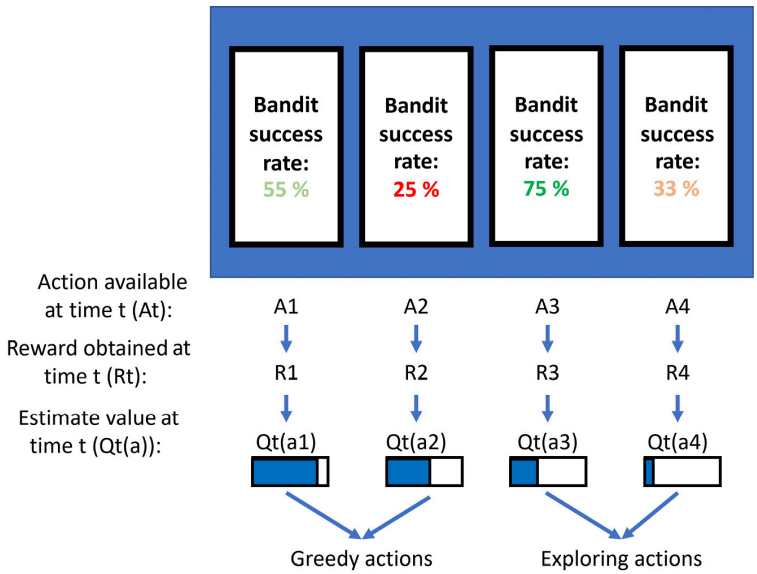


Figure 8.5 – Multi-arm bandit

Exploring may bring a decrease in gain in later steps but guarantees a greater gain in the long run. This is because our estimation may not be correct. Exploring allows us to correct the estimated value for an action. Especially in the early steps, it is more important to explore so that the system can understand which actions are best. In the final steps, the model should exploit the best actions. For this, we need a system that allows us to balance exploration toward exploitation.

To get an initial estimate of value, we can take an average of the rewards that are received:

$$Q_t\,a) = \frac{sumofrewardswhentheactionaistakenpriortotimestept}{numberoftimestheactionaistakenpriortotimestept} = \frac{\sum_{i=1}^{t-1} R_i \cdot 1_{\{A_i=a\}}}{\sum_{i=1}^{t-1} 1_{\{A_i=a\}}}$$

In this simple equation, *1* represents a variable that indicates whether the action was used at the time step (1 if used, 0 if not used).

$$1_{\{A_i=a\}} = \begin{matrix} 1if actionaistakenattimestepi \\ 0otherwise \end{matrix}$$

If the action has never been used, the denominator would be zero; to avoid the result being infinite, we use a default value (e.g., 0). If the number of steps goes to infinity, the estimated value should converge to the true value. Once these estimated values are obtained, we can choose the action. The easiest way to select an action is to choose the highest value (greedy action). The **`arg max`** function does exactly that:

$$A_t = arg\max_a Q_t\,a)$$

As we said before, we don't always want to choose greedy actions, but we want the model to explore other actions as well. For this, we can introduce a probability ε, so that the agent will select from the other actions with equal probability. In simple words, the model almost always selects the greedy action, but with a probability ε, it selects one of the other actions (regardless of its value). By increasing the number of steps, the other actions will also be tested (at infinity, they will be tested an infinite number of times) ensuring the convergence of *Q* to *q**. Similarly, the probability of selecting the best action converges to *1 - ε*. This method is called the **ε-greedy** method, and it allows for some balancing between exploitation and exploration.

To give a simple example, we can imagine a 10-armed bandit (*k*=10) where we have action values *q**, where we have a normal distribution to represent the true value of each action. Here, we have plotted 1,000 examples:

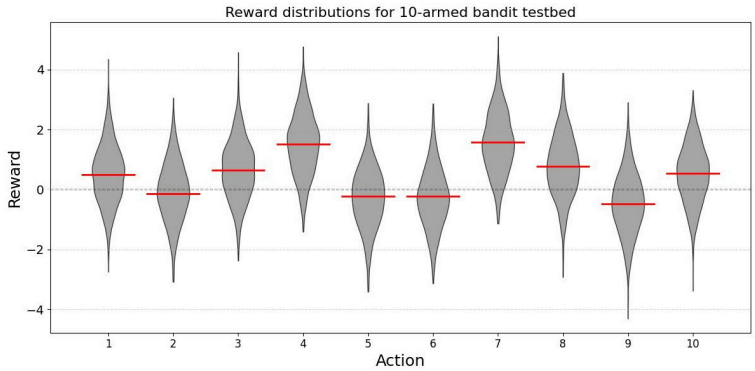


Figure 8.6 – Action value distribution

In the following example, we compare different ε-greedy methods. The rewards increase with agent experience and then go to plateaus. The pure greedy method is suboptimal in comparison to methods that also allow exploration. Similarly, choosing an ε constant that is too high (ε=0.5) leads to worse results than a pure greedy method.

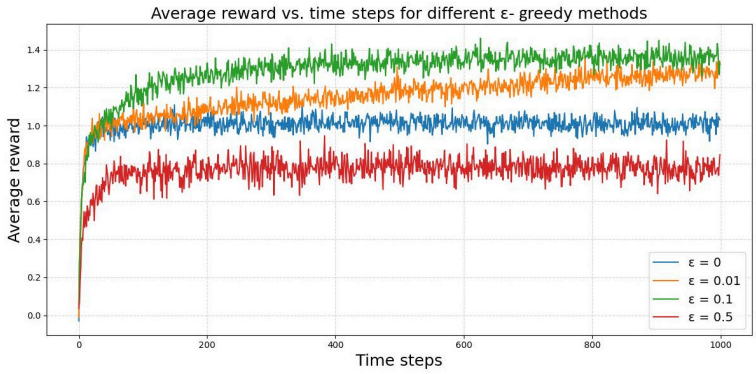


Figure 8.7 – Average rewards for time step for different greedy methods

To investigate this phenomenon, we can look at the optimal choice by the agent (*Figure 8.8*).

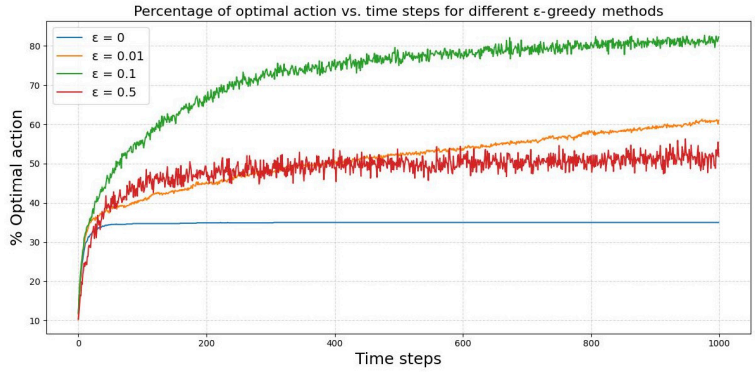


Figure 8.8 – Percentage optimal choice for time step for different greedy methods

The greedy method chooses the optimal choice only one-third of the time, while a method that includes some exploration selects the optimal choice 80% of the time (ε=0.1)%. This result shows that an agent that can also explore the environment achieves better results (can recognize the optimal action), while an agent that is greedy in the long run will choose actions that are suboptimal. In addition, ε-greedy methods find the optimal action faster than greedy methods.

In this case, we explored simple methods, where ε is constant. In some variants, ε decreases with the number of steps, allowing the agent to shift focus from exploration to exploitation once the environment has been sufficiently explored. ε-greedy methods work best in almost all cases, especially when there is greater uncertainty (e.g., greater variance) or when the system is nonstationary.

The system we have seen so far is not efficient when we have a large number of samples. So, instead of taking the average of observed rewards, we can use an incremental method (the one most widely used today). For an action selected i times, the reward will be R_i, and we can calculate the estimated value Q_n as:

$$Q_n = \frac{R_1 + R_2 + \cdots + R_n}{n-1}$$

At this point, we do not have to recollect the average each time but can keep a record of what was calculated and simply conduct an update incrementally in this way:

Qn+1=Qn+1n[Rn-Qn]

This can be seen simply as a kind of stepwise adjustment of the expected value:

$$Estimated_{new} \leftarrow Estimated_{new} + step_size[Target - Estimated_{old}]$$

In fact, we can see [*Target - estimated*$_{\text{old}}$] as a kind of error in the estimation that we are trying to correct step by step and bring closer to the real target. The agent is trying to move the value estimation to the real value.

We can test this incremental implementation, and we can see how, after an initial exploratory phase, the agent begins to exploit optimal choice:

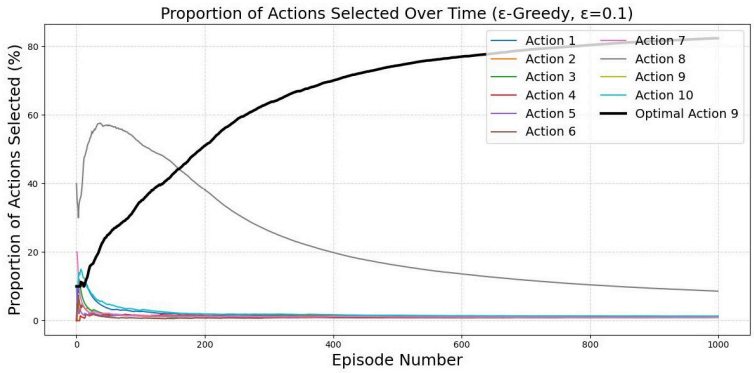


Figure 8.9 – Incremental implementation

1/n can be replaced by a fixed step size parameter α.

Qn+1=Qn+α[Rn-Qn]

Using α not only simplifies the calculation but also reduces the bias inherent in this approach. In fact, the choice of the initial value estimation for an action, $Q_{1(a),}$, can significantly influence early decisions and convergence behavior α also allows you to handle non-stationary problems better (where the reward probability changes over time). The initial expected values are in general set to 0, but choose values greater than 0. This alternative is called the optimistic greedy strategy; these optimistic values stimulate the agent to explore the environment more (even when we use a pure greedy approach with ε=0). The disadvantage is that we have to test different values for the initial *Q*, and in practice, almost all practitioners set it to 0 for convenience.

By testing the optimistic greedy method, we can see that it behaves similarly to the ε-greedy method:

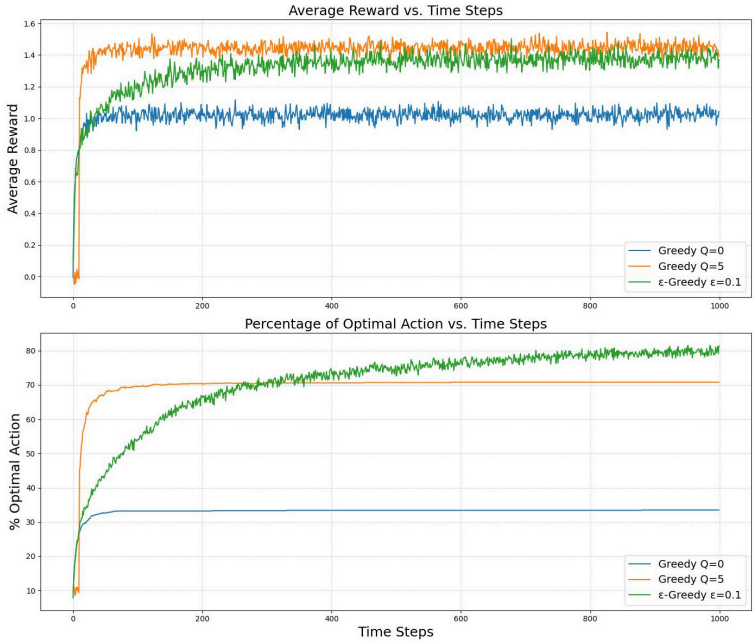


Figure 8.10 – Optimistic greedy versus the ε-greedy method

In non-stationary problems, α can be set to give greater weight to recent rewards than to prior ones.

One final note: so far we have chosen greedy actions for their higher estimated value. In contrast, we have chosen non-greedy actions randomly. Instead of choosing them randomly, we can select them based on their potential optimality and uncertainty. This method is called **Upper Confidence Bound** (**UCB**), where an action *A* is selected based on:

$$A_t = arg\max_a \left[Q_t\, a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

where *ln*(*t*) represents the natural logarithm of *t*, *c*> *0* controls exploration, and N_t is the number of times an action *A* has been tested. This approach means that all actions will be tested but actions that have a lower estimate value (and have been tested frequently) will be chosen again in a decreasing manner. Think of it as choosing between different restaurants. UCB helps you balance between going to the one you already like (high estimated value) and trying out others that might be better but haven't visited much yet (high uncertainty). Over time, it naturally reduces exploration of poorly performing options while continuing to test under-explored but potentially good ones. UCB works very well, though it is difficult to apply in approaches other than multi-armed bandits.

As you can see, UCB generally gives better results:

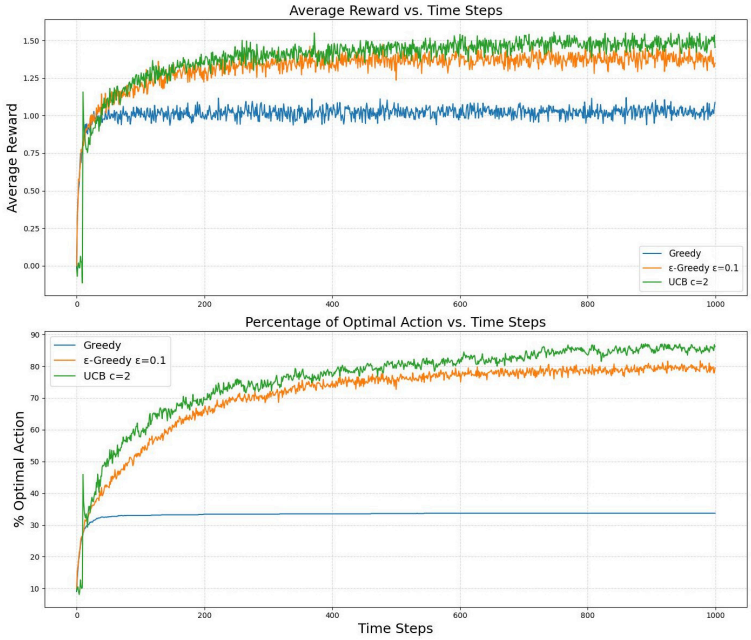


Figure 8.11 – UCB improvements on greedy methods

Multi-armed bandit is a classic example of RL, but it allows one to begin to understand how RL works.

Multi-armed bandit has been used for several applications, but it is a simplistic system that cannot be applied to different real-world situations. For example, in a chess game, the goal is not to eat the chess pieces but to win the game. Therefore, in the next subsection, we will begin to look at methods that take into account a purpose farther back in time than immediate gain.

Markov decision processes

Markov decision processes (MDPs) are problems where actions impact not only immediate rewards but also future outcomes. In MDPs then, delayed reward has much more weight than what we saw in the multi-armed bandit problem, but also in deciding the appropriate action for different situations.

Imagine you are navigating a maze. Each intersection or hallway you enter is a state, and every turn you make is an action that changes your state. Some paths lead you closer to the exit (the final reward), while others might take you in circles or into dead ends. The reward for each move may not be immediate—you only get the big reward when you reach the end. So, every action you take needs to consider how it impacts your chances of reaching the goal later on.

In MDPs, this idea is formalized: the agent must decide the best action in each state, not just for instant rewards, but for maximizing long-term success, which makes them more complex than simpler problems such as the multi-armed bandit, where only immediate rewards are considered.

Previously we were just estimating $q^*(a)$. Now, we want to estimate the value of action a in the presence of state s, $q^*(s,a)$. At each time step, the agent receives a representation of the environment S_t and performs an action A_t, receives a reward R, and moves to a new state S_{t+1}. It can be seen that the agent's action can change the state of the environment.

In a finite MDP, the set of states, actions, and rewards contains a finite number of elements. The variables *R* and *S* are probability distributions that depend on both the previous state and the action. We can describe the dynamics of this system using the state-transition probability function *p(s', r | s, a)*:

$$p\ s\prime, r|s, a) = \Pr\ S_t = s\prime, R_{t=r}|S_{t-1} = s, A_{t-1} = a\}$$

In other words, state and reward depend on the previous state and the previous action. At each time step, the new state and reward will derive from the previous cycle. This will repeat a finite series of events. So, each state will summarize all the previous information (for example, in tic-tac-toe, the new system configuration gives us information about the previous moves) and is said to be a Markov state and possess the Markov property. The advantage of a Markov state is that each state possesses all the information we need to predict the future. The preceding function describes to us how one state evolves into another as we perform actions. An RL problem that respects this property is called an MDP. So, from this function, we can derive anything we care about the environment. We can then derive state-transition probabilities:

$$p\ s\prime|s, a) = \Pr\ S_t = s\prime|S_{t-1} = s, A_{t-1} = a\Big\} = \sum_{r\in R} p\ s\prime, r|s, a)$$

We can also derive the expected reward for state-action pairs:

rs,a=ERtSt-1=s,At-1=a=∑r∈Rr∑s∈Sps',rs,a

And we can derive the expected rewards for state-action-next-state triples:

$$r\ s, a, s\prime) = \mathbb{E}\ R_t|S_{t-1} = s, A_{t-1} = a, S_t = s\prime] = \sum_{r\in R} r\frac{p\ s\prime, r|s, a)}{p\ s\prime|s, a)}$$

This shows us the flexibility of this framework. A brief note is that *t* does not need to be time steps but a sequence of states (a series of decisions, movements of a robot, and so on) making MDP a flexible system. After all, in MDPs, every problem can be reduced to three signals: actions, states, and rewards. This allows us a better abstraction to represent goal-oriented learning.

The goal for an agent is clearly to maximize cumulative reward over a long period of time (rather than immediate gain). This system has been shown to be very flexible because much of the problem can be formalized in this way (one just has to find a way to define what the rewards are so that the agent learns how to maximize them). One note is that agents try to maximize the reward in any way possible; if the goal is not well defined, it can lead to unintended results (e.g., in chess, the goal is to win the game; if the reward is to eat a piece and not to win the game, the agent will try to maximize the pieces eaten even when it might lead to losing the game).

This can be better expressed formally, where *G* is the cumulative sum of rewards received from step *t* and later:

$$G_t = R_{t+1} + R_{t+2} + \ldots + R_T$$

Our goal is thus to maximize *G*. This is easier to define for a sequence where we clearly must have an end (e.g., a game where only a defined number of moves can be made). A defined sequence of steps is called an episode, and the last state is called a terminal state. Note that each episode is independent of the other (losing one game does not affect the outcome of the next). This is, of course, not always possible; there are also definite continuous tasks where there is no net end (a robot moving in an environment), for which the previous equation does not work. In this case, we can use a so-called discount rate γ. This parameter allows us to decide the agent's behavior: when γ is near 0, the agent will try to maximize immediate rewards, while approaching 1, the agent considers future rewards with greater weight:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

As we saw in the *multi-armed bandit problem* section, we can estimate value functions (how good it is for an agent to be in a given state). The value of a state *S* considering a policy π, is the expected return starting from *S* and following the policy π from that time:

$$v_\pi s) = \mathbb{E}_\pi G_t | S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right] for all s \in S$$

This is called the state-value function for the policy π, and G_t is the expected return. Similarly, we can define the value of taking action *A* in state *S* under policy π:

$$q_\pi s, a) = \mathbb{E}_\pi G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a \right]$$

This is called the action-value function for policy π. We can estimate these functions by experience (interaction with the environment), and at infinity, they should approach the true value. Methods like this where we conduct averaging of many random samples of actual returns are called Monte Carlo methods.

For efficiency, we can rewrite it in a recursive form (using discounting):

$$v_\pi s) = \mathbb{E}_\pi G_t | S_t = s] = \mathbb{E}_\pi R_{t+1} + \gamma G_{t+1} \middle| S_t = s] = \sum_a \pi(a|s) \sum_{s\prime,r} p\, s\prime, r|s, a)\, r + \gamma v_\pi s\prime)] for all s \in S$$

This simplified form is called the Bellman equation. This can be represented as thinking forward to the next state from the previous state. From a state with a policy, we choose an action and get a reward (or not) with a given probability. The Bellman equation conducts the average of these probabilities, giving a weight to the possibility of their occurrence. This equation is the basis of many great RL algorithms.

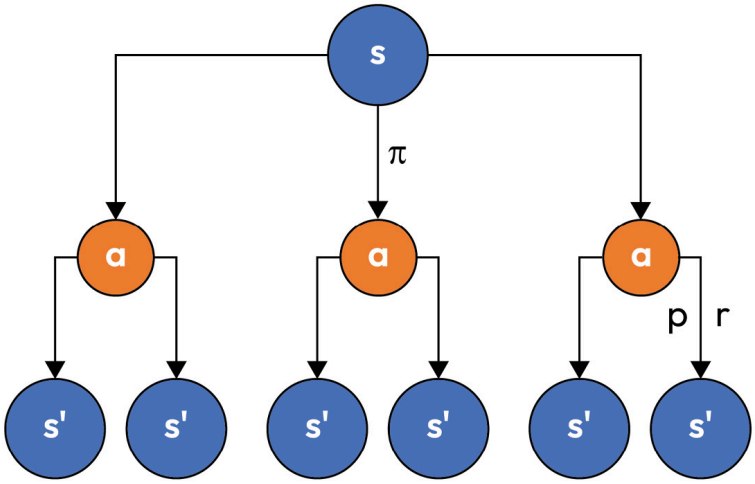


Figure 8.12 – Bellman backup diagram

Now that we have both state-value functions *vπ(s)* and action-value functions *qπ(s, a)*, we can evaluate policies and choose the best ones. Action-value functions allow us to choose the best action relative to the state. Consider, for example, a case of a Texas Hold'em poker game. A player has $100 and must choose the strategy starting from the state π. The strategy π_1 has a state value function that returns 10, while π_2 has a return of -2. This means that the first strategy brings an expected gain of 10, while π_2 brings an expected loss of 2. Given a state *s*, the player wants to figure out which action to choose. For example, choosing whether to bet 10 or 5, q_π (s, a) tells us what the expected cumulative reward is from this action. So, the preceding equations allow us to figure out which action or strategy to choose to maximize the reward.

From *Figure 8.12*, it can be understood that solving an RL task means finding an optimal policy that succeeds in collecting many rewards over the long run. For MDPs, it is possible to define an optimal policy because we can evaluate whether one policy is better than another if it has a higher expected return for all states *vπ(s)*. π^* denotes the optimal policy and is the one that has the maximum value function over all possible policies:

$$v_* s) = \max_\pi v_\pi s) forall s \in S$$

The optimal policies share the same optimal action-value function *q**, which is defined as the maximum action-value function over all possible policies:

$$q_* s, a) = \max_\pi q_\pi s, a) forall s \in S$$

The relationship between these two functions can be summarized as follows:

$$q_* s, a) = \mathbb{E}\, R_{t+1} + \gamma v_*(S_{t+1}) \Big| S_t = s, A_t = a]$$

The equation expresses the cumulative return given a state-action pair. Optimal value functions are an ideal state in RL, though, and it is difficult to find optimal policies, especially when tasks are complex or computationally expensive. RL therefore tries to approximate them, for example, by using **dynamic programming** (**DP**). The purpose of DP is to use value functions to search for good policies (even if not exact solutions). At this point, we can derive Bellman optimality equations for the optimal state-value function *v*(s)* and the optimal action-value function q_* *(s, a)*:

$$v_* s) = \max_a \mathbb{E}\, R_{t+1} + \gamma v_*(S_{t+1}) \Big| S_t = s, A_t = a] = \max_a \sum_{s\prime,r} p(s\prime, r|s, a) \Big[r + \gamma v_* + s\prime \Big]$$

$$v_* s) = \max_a \mathbb{E}\, R_{t+1} + \gamma v_*(S_{t+1}) \Big| S_t = s, A_t = a] = \max_a \sum_{s\prime,r} p(s\prime, r|s, a) \Big[r + \gamma v_* + s\prime \Big]$$

$$q_* s, a) = \mathbb{E}\, R_{t+1} + \max_{a\prime} q^* \Big(S_{t+1}, a\prime \Big) \Big| S_t = s, A_t = a \Big] = \sum_{s\prime,r} p(s\prime, r|s, a)\, r + \gamma \max_{a\prime} q^* \Big(s\prime, a\prime \Big) \Big]$$

$$q_* s, a) = \mathbb{E}\, R_{t+1} + \max_{a\prime} q^* \Big(S_{t+1}, a\prime \Big) \Big| S_t = s, A_t = a \Big] = \sum_{s\prime,r} p(s\prime, r|s, a)\, r + \gamma \max_{a\prime} q^* \Big(s\prime, a\prime \Big) \Big]$$

For finite MDP, Bellman optimality equations have only one solution, and they can be solved if we know the dynamics of the system. Once we get v^*, it is easy to identify the optimal policy q^*; having an optimal q^*, we can identify the optimal actions. The beauty of v^* is that it allows us to choose the best actions at the moment while still taking into account the long-term goal. Solving these equations for a problem is solving the problem through RL. On the other hand, for many problems, solving them means calculating all possibilities and thus would be too computationally expensive. In other cases, we do not know the dynamics of the environment with certainty or the states do not have Markov properties. However, these equations are the basis of RL, and many methods are approximations of these equations, often using experience from previous states. So, these algorithms do not identify the best policy but an approximation. For example, many algorithms learn optimal actions for the most frequent states but may choose suboptimal actions for infrequent or rare states. The trick is that these choices should not impact the future amount of reward. For example, an agent might still win a game even if it does not make the best move in rare situations.

DP refers to a collection of algorithms that are used to compute the best policy given a perfect model of the environment as an MDP. Now, these algorithms require a lot of computation and the assumption of the perfect model does not always hold. So, these algorithms are not practically used anymore; at the same time, one can define today's algorithms as inspired by DP algorithms, with the purpose of reducing computation and working even when the assumption of a perfect model of the environment does not hold. DP algorithms, in short, are obtained from transforming Bellman equations into update rules to improve the approximation of desired value functions. This allows value functions to be used to organize the search for good policies. To evaluate a policy, we can use the state-value function and evaluate the expected return when following policy π from each state:

$$v_\pi s) = \mathbb{E}\, R_{t+1} + \gamma v_\pi(S_{t+1}) \Big| S_t = s, A_t = a] = \sum_a \pi(a|s) \sum_{s\prime,r} p(s\prime, r|s, a) \Big[r + \gamma v_\pi \Big(s\prime \Big) \Big]$$

Calculating the value function for a policy aims to identify better policies. For a state s, we want to know whether we should keep that policy, improve it, or choose another. Remember that the choice of a policy decides what actions an agent will take. To answer the question "is it better to change policy?", we can consider what happens if we choose an action in a state s following policy π:

$$q_\pi s, a) = \mathbb{E}\, R_{t+1} + \gamma v_\pi(S_{t+1}) \Big| S_t = s, A_t = a] = \sum_{s\prime,r} p(s\prime, r|s, a) \Big[r + \gamma v_\pi \Big(s\prime \Big) \Big]$$

A better policy π' should provide us with a better value of $v_{\pi}(s)$. If π' is less than or equal to $v_{\pi}(s)$, we can continue the same policy. In other words, choosing actions according to a policy π' that has better $v_{\pi}(s)$ is more beneficial than another policy π.

In this section, we have seen classic RL algorithms, but none of them use a neural network or other machine learning model. These algorithms work well for simple cases, while for more complex situations we want a more sophisticated and adaptable system. In the next section, we will see how we can integrate neural networks into RL algorithms.

Deep reinforcement learning

Deep reinforcement learning (**deep RL**) is a subfield of RL that combines RL with deep learning. In other words, the idea behind it is to exploit the learning capabilities of a neural network to solve RL problems. In traditional RL, policies and value functions are represented by simple functions. These methods work well with low-dimensional state and action spaces (i.e., when the environment and agent can be easily modeled). When the environment becomes more complex or larger, traditional methods fail to generalize. In deep RL, instead, policies and value functions are represented by neural networks. A neural network can theoretically represent any complex function (Universal Approximation Theorem), and this allows deep RL methods to solve problems with high-dimensional state spaces (such as those presenting images, videos, or continuous tasks). Modeling complex functions thus allows the agent to learn a more generalized and flexible policy that is needed in complex situations where defining a function is impossible with traditional methods. This learning capability has enabled deep RL methods to solve video games, move robots, and more.

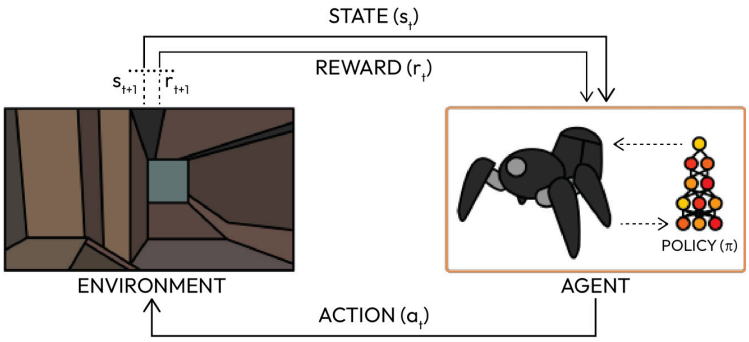


Figure 8.13 – Overview of deep RL (https://arxiv.org/abs/1708.05866)

In the upcoming subsections, we will discuss how to classify these algorithms and what the differences are.

Model-free versus model-based approaches

There are so many methods of deep RL today that it is difficult to make a taxonomy of these models. Nevertheless, deep RL methods can be broadly divided into two main groups: model-free and model-based. This division is represented by the answer to this question: does the agent have access to (or learn) a model of the environment?

- **Model-free methods**: These methods determine the optimal policy or value function without building a model of the environment. These models learn directly from observed states, actions, and rewards. The agent learns directly from trial and error, receives feedback from the environment, and uses this feedback to improve its policy or value estimation. These approaches are usually easier to implement and conduct parameter tuning (they only require observing state-action-reward sequences or transitions). They are also more easily scalable and less computationally complex.
- **Model-based methods**: These methods rely on an internal model of the environment to predict future states and rewards given any state-action pair. This model can be learned or predefined before training. Having the model allows the agent to have similar outcomes and plan actions for future scenarios (e.g., what the future actions of an opponent in a game will be and anticipating them). Model-based approaches have the advantage that they can reduce interaction with the real environment and are better at planning complex tasks. Potentially improved performance comes at the cost of increased complexity (building an accurate model of the environment can be challenging, especially for high-dimensional environments) and increased computational cost.

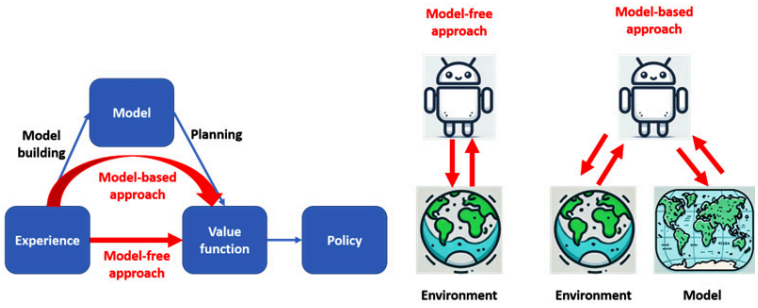


Figure 8.14 – Model-free versus model-based approaches

The primary advantage of a model-based RL method lies in its ability to plan and think ahead. By utilizing a model (in general, a neural network) to simulate the dynamics of the environment, it can predict future scenarios, making it particularly useful in complex environments or situations where decisions need to consider long-term outcomes. For instance, when rewards are sparse or delayed—such as in chess, where the reward is achieved only by winning the game—the model can simulate various paths to optimize the agent's strategy for reaching the reward.

Planning also proves advantageous in dynamic environments. The model can update its internal representation quickly, allowing the agent to adapt its policy without relearning from scratch. This minimizes the need for extensive retraining, as seen in applications such as autonomous driving, where the agent can adjust its strategy without requiring large new datasets. The insights gained from such planning can then be distilled into a learned policy, enhancing the agent's performance over time.

Additionally, simulating interactions with the environment reduces the need for extensive real-world exploration, which is critical in scenarios where interactions are costly, dangerous, or time-intensive, such as in robotics or autonomous vehicles. By leveraging its internal model, the agent can prioritize actions and refine its exploration process to update or improve its understanding of the environment more efficiently.

This then helps the agent optimize for long-term goals because it can simulate the long-term consequences of its actions, monitor its progress toward a more distant horizon, and align its actions with a distant goal.

Model building can be a complicated task. However, a ground-truth model of the environment is not always available to the agent. In this case, the agent is forced to learn only from experience to create its own model. This can then lead to bias in the agent's model. An agent might therefore perform optimally with respect to a learned model but perform terribly (or suboptimally) in the real environment.

On-policy versus off-policy methods

Another important classification in RL is how models learn from experience and whether they learn from the current policy or a different one (they are classified according to the relationship between the policy and the policy update):

- **On-policy methods**: These methods learn from actions learned from the agent's current policy (so the agent both collects data and learns from the same policy). On-policy methods evaluate and improve the policy used to make decisions; this is based on actions taken and rewards received while following the current policy (the agent conducts the policy update by directly evaluating and improving the policy). The agent therefore does not use data from other policies. The advantages are that the agent tends to be more stable and less prone to variance (the optimized policy is, in fact, the one used to interact with the environment). On-policy methods are inefficient because they discard data that is obtained from previous policies (sample inefficiency), limiting their use for complex environments since they would require large amounts of data. In addition, these methods are not very exploratory and therefore less beneficial where more exploration is required (they are favored for environments that are stable). An example is a chatbot that learns to give better answers to user questions: the chatbot uses a specific policy to give answers and optimizes this policy by leveraging feedback received from users. On-policy methods ensure that the learned policy is linked to actions taken by the chatbot and by real interactions with users (this ensures stability).
- **Off-policy methods**: Off-policy methods learn the value of the optimal policy independently of the agent's actions (agents learn from experiences that are generated by a different policy from the one used for learning). So, these methods can learn from past data or data that is generated by other policies. Off-policy methods separate the behavior policy (used to collect data) from the target policy (the policy being learned). In other words, the behavior policy is used to explore the environment while the target policy is used to improve the agent's performance (to ensure more exploratory behavior while learning an optimal target policy). Off-policy methods have higher sample efficiency because they can reuse data and allow for better exploration, which can lead to faster convergence to an optimal policy. On the other hand, they are less stable (because they do not learn from actions that have been taken by the current policy, the discrepancy between behavior policy and target policy can lead to higher variance in updates) and can be much more complex. An example is a music recommendation system that suggests new titles to users and has to explore different genres and new releases. The behavior policy encourages exploration

and thus generates data on user preferences, while the target policy seeks to optimize recommendation performance for users. Separating the two policies thus allows experimenting with different recommendation strategies without compromising the quality of the final recommendations. The advantage of these methods is that they allow extensive exploration, which is very useful for complex and dynamic environments.

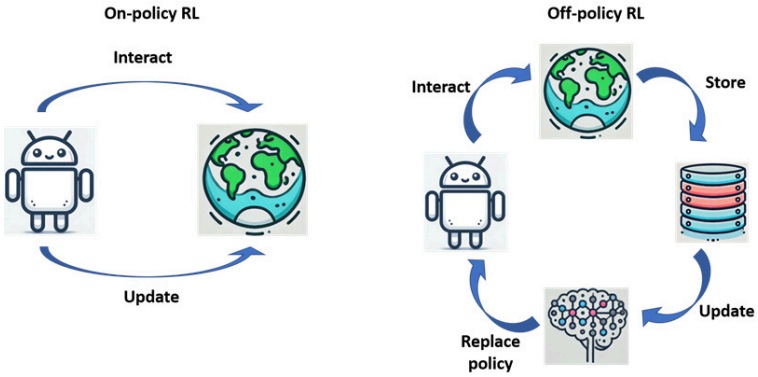


Figure 8.15 – On-policy and off-policy methods

In the next subsection, we will begin to go into detail about how deep RL works.

Exploring deep RL in detail

We'll begin with a definition to better understand deep RL. A state *s* in a system is usually a vector, matrix, or other tensor. At each time step *t*, we can describe the environment in the form of a tensor (e.g., the position of the pieces on a chessboard can be represented by a matrix). Similarly, the actions *a* an agent can choose can be represented in a tensor (for example, each action can be associated with a one-hot vector, a matrix, and so on). All of these are data structures that are already commonly seen in machine learning and that we can use as input to a deep learning model.

So far, we have discussed policies generically, but what functions do we use to model them? Very often, we use neural networks. So, in this section, we will actively look at how a neural network can be used in RL algorithms. What we'll see now is based on what we saw in this chapter, but we'll use a neural network to decide what action to take (instead of just a function). As we saw in ***Chapter 1***, a neural network is constituted of a series of neurons organized in a series of layers. Neural networks take a tensor as input and produce a tensor as output. In this case, the output of the neural network is the choice of an action. Optimizing the policy, in this case, means optimizing the parameters of the neural network. An RL algorithm based on experience can change the parameters of the policy function so that it produces better results.

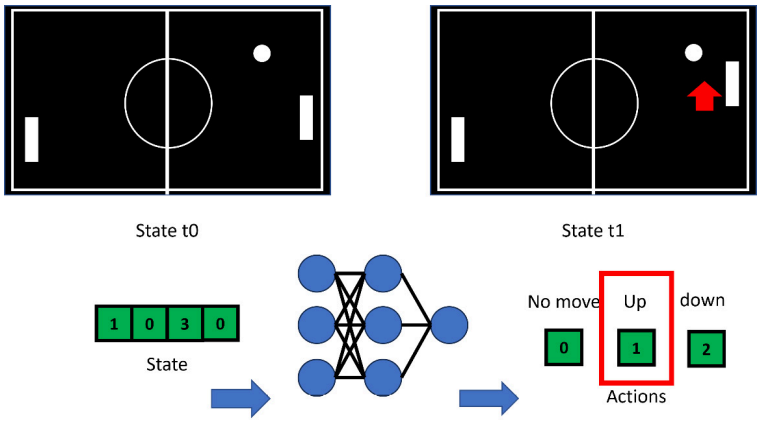


Figure 8.16 – Neural network as an RL policy

Neural networks are well-known deep learning models, and we know how to optimize them. Using gradient-based methods allows us to understand how a change in parameters impacts the outcome of a function. In this case, we want to know how we should update the parameters of our policy *P* (the neural network model) so that we collect more rewards in the future. So, having a function that tells us what the expected rewards are for a policy, we can use the gradient to change the parameters of the policy and thus maximize the return.

Using a neural network as a policy in RL has several advantages:

- Neural networks are highly expressive function approximators, so they can map complex nonlinear relationships between inputs (states) and outputs (actions). This is very useful for complex environments, such as playing video games or controlling robots in 3D environments. In addition, neural networks scale well for environments that have large and complex state and action spaces.
- Neural networks possess the ability to generalize to situations they have not encountered before. This capability makes them particularly useful in handling unexpected state changes, thus promoting adaptability in agents. All this allows neural networks to be flexible and adaptable to a different range of tasks and environments.
- Neural networks can handle actions that are continuous rather than discrete, thus enabling their use in real-world problems (where actions are often not limited to a discrete set).
- Neural networks are versatile. They can be used with different types of data and do not require feature engineering. This is important when the features can be complex or the state representation is complex (sensors, images, video, and so on).
- They can produce a probability distribution and thus can be used with a stochastic policy. This is important when we want to add randomness and encourage exploration.



Figure 8.17 – Examples of screenshots where a neural network is used to learn how to play Atari games (https://arxiv.org/abs/1312.5602)

We will now present five different algorithms in order to understand the differences between the different types of approaches (off- and on-policy, model-free, and model-based approaches).

Algorithm	On-policy	Off-policy	Model-free	Model-based
Deep Q-Network (DQN)		yes	yes	
REINFORCE	yes		yes	
Proximal Policy Optimization (PPO)	yes		yes	
Actor critic	yes		yes	
AlphaZero		yes		yes

Figure 8.18 – Summary table of RL approaches

Q-learning and Deep Q-Network (DQN)

Q-learning is a lookup-table-based approach underlying **Deep Q-Network** (**DQN**), an algorithm used by DeepMind to train an agent capable of solving video games. In the Q-learning algorithm, we have a **Q-table of State-Action values**, where we have a row for each state and a column for each action, and each cell contains an estimated Q-value for the corresponding state-action pair. The Q-values are initially set to zero. When the agent receives feedback from interacting with the environment, we iteratively conduct the update of the values (until they converge to the optimal values). Note that this update is conducted using the Bellman equation (the Q-value in the table represents the expected future rewards if the agent takes that action from that state and follows the best strategy afterward).

Q-learning finds the optimal policy by learning the optimal Q-value for each state-action pair. Initially, the agent chooses actions at random, but by interacting with the environment and receiving feedback (reward), it learns which actions are best. During each iteration, it conducts the table update using the Bellman equation. The agent generally chooses the action that has the highest Q-value (greedy strategy), but we can control the degree of exploration (ε-greedy policy). Over time, these estimates become more and more accurate and the model converges to the optimal Q-values.

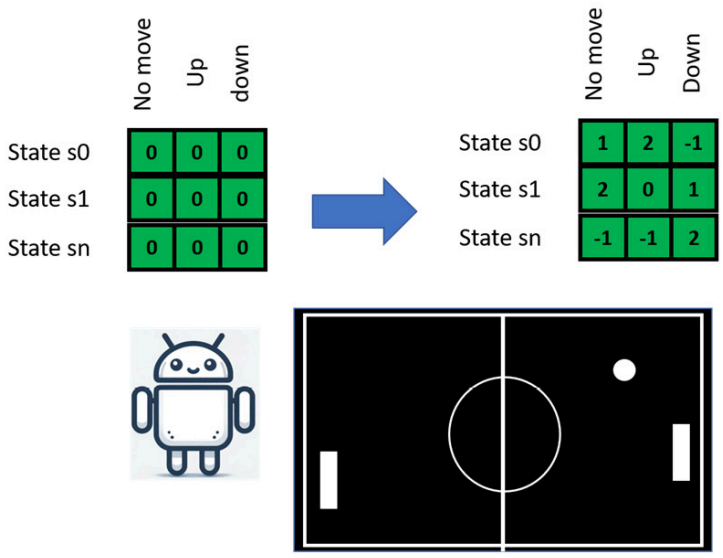


Figure 8.19 – Q learning example

In complex environments, using a table to store values becomes impractical due to the potentially massive size and computational intractability. Instead, we can use a Q-function, which maps state-action pairs to Q-values. Given that neural networks can effectively model complex functions, they can be employed to approximate the Q-function efficiently. By providing as input the state *S*, the neural network provides as output the Q-

value for the state-action pair (in other words, the Q-values for all the actions you can take from that state). The principle is very similar to the Q-learning algorithm. We start with random estimates for the Q-values, explore the environment with an ε-greedy policy, and conduct the update of the estimates.

The DQN architecture consists of three main components: two neural networks (the Q-network and the target network) and an experience replay component. The Q-network (a classical neural network) is the agent that is trained to produce the optimal state-action value. Experience replay, on the other hand, is used to generate data to train the neural network.

The Q-network is trained on multiple time steps and on many episodes, with the aim of minimizing the difference between predicted Q-values and the target Q-values. During the agent's interaction with the environment, each experience (a tuple of state, action, reward, and next state) is stored in this experience replay buffer. During training, random batches of experiences (a mix of old and new experiences) are selected from the buffer to update the Q-network. This allows breaking the correlation between consecutive experiences (helps stabilize the training) and reusing the past experience multiple times (increases data efficiency). The target network is a copy of the Q-network used to generate the target Q-values for training. Periodically, the target network weights are updated (e.g., every few thousand steps) by copying the Q-network weights; this stabilizes the training. During training, the Q-network predicts the Q-value for actions given a state (predicted Q-value) and the target network predicts the target Q-value for all actions given the state. The predicted Q-value, target Q-value, and the observed reward are used to calculate the loss and update the weight of the Q-network.

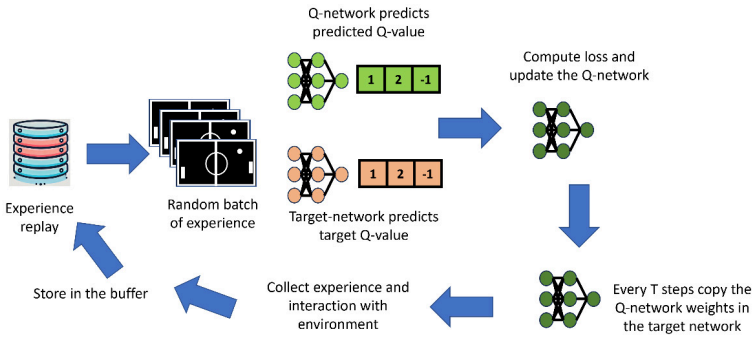


Figure 8.20 – DQN training algorithm

DQN has a number of innovations and advantages:

- Experience replay makes training more stable and efficient. Neural networks usually take a batch of data as input rather than a single state, so during training, the gradient will have less variance and the weights converge more quickly. Experience replay also allows us to reduce noise during training because we can conduct a kind of "shuffling" of experiences and thus better generalization.
- The introduction of a target network mitigates the issue of non-stationary targets, which can cause instability in training. The target network is untrained, so the target Q-values are stable and have few fluctuations.
- DQN is effective with high-dimensional spaces such as images and is able to extract features by itself and learn effective policies. These ca-

pabilities enabled DQN to master Atari games by taking raw pixels as input.

There are, of course, also drawbacks:

- Although it is more efficient than Q-learning, DQN still requires a large number of samples to learn effectively; this limits its use for tasks where there is little data (sample inefficiency)
- It is not stable when the action spaces are continuous, while it works well for discrete action spaces
- It is sensitive to the choice of hyperparameters (such as learning rate, replay buffer size, and update frequency of the target network)

The REINFORCE algorithm

DQN focuses on learning the value of an action in different states. **REINFORCE** is instead a policy-based method. These methods learn policy directly, mapping states to actions without learning a value function. The core idea is to optimize policy by maximizing the expected cumulative reward the agent receives over time. REINFORCE is a foundational algorithm for learning how to train agents to handle complex, continuous action spaces.

The policy is represented by a neural network that takes the current state as input and produces a probability distribution over all possible actions (the probability that an agent will perform a certain action). This is called stochastic policy because we do not have an action as output directly, but probabilities. Policy gradient methods try to improve the policy directly (by changing parameters during training) so that the policy produces better results. So, again, we start with a random policy (the neural network weights are initialized randomly) and let the agent act in the environment according to its policy, which causes a trajectory (a series of states and actions) to be produced. If this trajectory collects high rewards, we conduct an update of the weights so that this trajectory is more likely to be produced in the future. If, on the contrary, the agent performs poorly, the update of the weights will be directed to make that trajectory less likely.

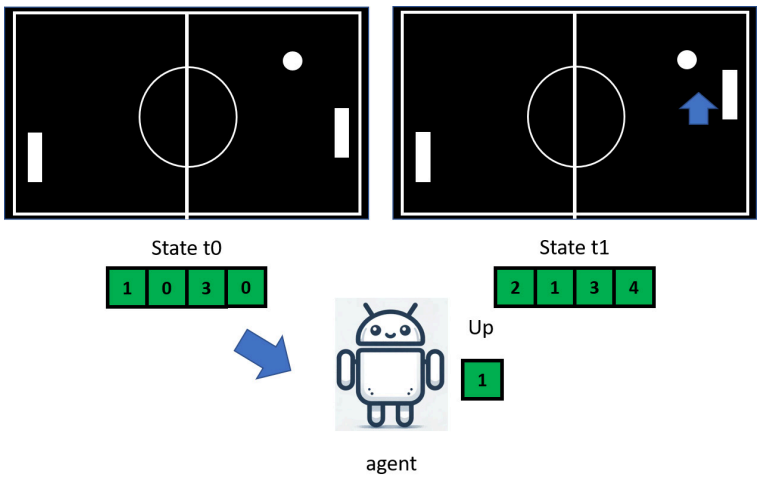


Figure 8.21 – Example of trajectory

So, the first step in this process is to initialize a neural network (policy P) with its parameters θ. Since these weights are initially random, the policy for a state as input will lead to random actions. We then generate a trajectory τ, letting the agent interact with the environment. Starting from state

s_0, we let the agent move according to policy P with the parameters θ. In practice, state S is given as input to the neural network and generates a distribution of actions. We select an action a_0 sampling from this distribution. This process is repeated for as long as possible (e.g., till the end of the game), the set of states and actions being our trajectory.

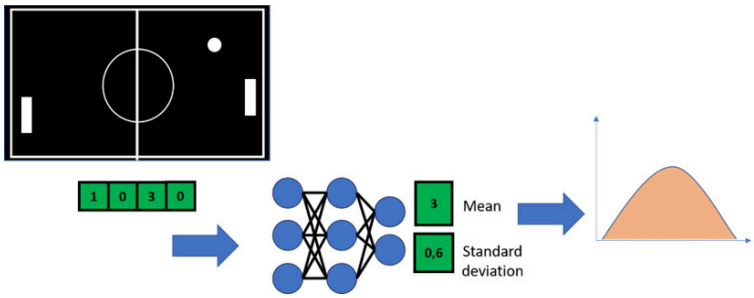


Figure 8.22 – Getting a distribution from a neural network

During the trajectory, we collect rewards (called reward-to-go or also return G_t). The return is the total cumulative reward received from time step t to the end of the episode, discounted by a factor γ. The discount factor determines how important future rewards are in relation to immediate rewards. In this case, we have a function that gives us the expected return for a given policy and we want to maximize it. So, we calculate the gradient, and via gradient ascent, we modify the parameters of our neural network.

REINFORCE is conceptually simple and easy to implement, suitable for continuous action spaces (since it directly learns a policy), and enables end-to-end learning (the algorithm learns directly from raw data). Some of the challenges with this algorithm, however, are the high variance in policy updates (it relies on full episodes being returned and updates can be noisy and unstable), it requires a lot of data (a large number of episodes are needed because it discards data after each update and does not reuse experiences), it is not suitable for environments where data is expensive, and it does not work well when there are delayed rewards.

Note that the REINFORCE algorithm is an on-policy algorithm since the policy receives updates only based on experiences collected with the same policy. At each iteration, the agent uses the updated policy and collects experience with it for the update. In the case of off-policy methods, experiences collected with other policies are also used. This is, for example, what we saw with DQN, where we were using experiences in the batch that were collected with a different policy.

Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is one of the most widely cited and used algorithms in RL. Introduced by OpenAI in 2017, PPO was designed to balance the simplicity of policy gradient methods, such as REINFORCE, with the stability of more complex algorithms, such as **Trust Region Policy Optimization (TRPO)**. In essence, PPO is a practical and efficient algorithm that performs well on benchmarks while being relatively easy to implement and tune.

PPO shares similarities with REINFORCE but includes important improvements that make training much more stable. One of the challenges in policy-based methods is the choice of hyperparameters (especially the learning rate) and the risk of unstable weight updates. The key innovation of

PPO is to ensure that policy updates are not too large, as this could destabilize training. PPO achieves this by using a constraint on the objective function that limits how much the policy can change in a single update, thereby avoiding drastic changes in the network weights.

A significant problem with traditional policy gradient methods is their inability to recover from poor updates. If a policy performs poorly, the agent may generate sparse or low-quality training data in the next iteration, creating a self-reinforcing loop that can be difficult to escape. PPO addresses this by stabilizing policy updates.

The policy in PPO is represented by a neural network, $\pi\theta(a|s)$, where θ represents the network's weights. The network takes the current state s as input and outputs a probability distribution over possible actions a. Initially, the weights are randomly initialized. As the agent interacts with the environment, it generates batches of experiences (state, action, reward) under the current policy. The agent also calculates the advantage estimate, which measures how much better (or worse) a chosen action was compared to the expected value of the state.

The main difference from simpler policy gradient methods lies in PPO's use of a **clipped objective function**. This function ensures that policy updates are stable and prevents large, destabilizing changes. If the probability ratio between the new and old policies $rt(\theta)$ falls outside the range $[1-\epsilon,1+\epsilon]$, where ϵ is a small hyperparameter (e.g., 0.2), the update is clipped. This clipping mechanism ensures that policy updates remain within a safe range, preventing the policy from diverging too much in a single update.

A common variant of PPO uses an **actor-critic architecture**, where the actor learns the policy, and the critic learns the value function. The critic provides feedback on the quality of the actions, helping to reduce the variance of the updates and improve learning efficiency (we discuss this more in detail later).

Overall, PPO is both a stable and robust algorithm, less prone to instability than simpler policy gradient methods and easier to use than more complex algorithms such as TRPO. It does not require solving complex optimization problems or calculating second-order gradients, making it a practical choice for many applications. However, PPO still requires careful tuning of hyperparameters, such as the clipping parameter ϵ, learning rate, and batch size. Additionally, it can suffer from high variance in environments with long episodes or delayed rewards.

The actor-critic algorithm

The actor-critic algorithm is another popular approach in RL, which combines the strengths of two different methods: value-based methods (such as Q-learning) and policy-based methods. The actor-critic model consists of two components:

- **Actor**: The actor is responsible for deciding what action should be taken in the current state of the environment. The policy is generally a neural network that produces a probability distribution over actions. The actor tries to maximize the expected return by optimizing the policy.

- **Critic**: The critic evaluates the actions taken by the actor by estimating the value function. This function indicates how good an action is in terms of expected future rewards. The value function can be the state value function *V(s)* or the action-value function *Q(s,a)*.

The insight behind this approach is that the actor is the decision-maker who learns how to improve decisions over time. The critic, on the other hand, is a kind of advisor who evaluates the goodness of actions and gives feedback on strategy.

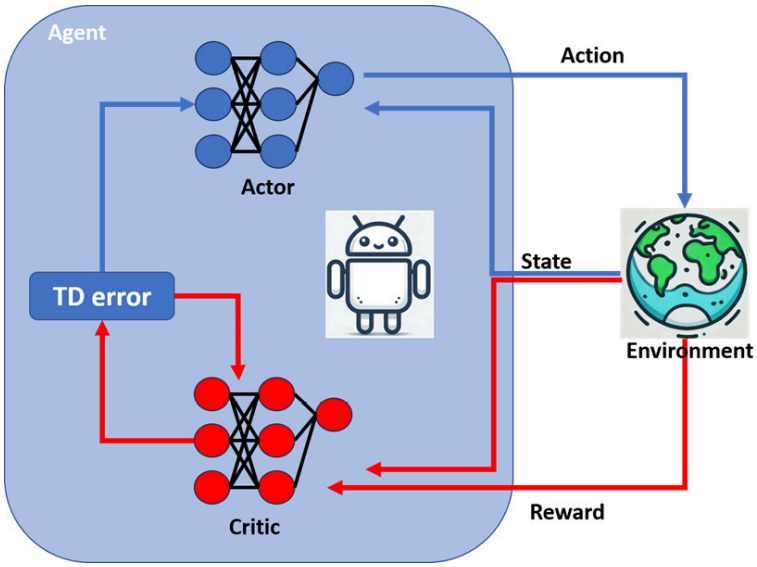


Figure 8.23 – Actor-critic approach

The process can be defined by four steps:

1. The agent interacts with the environment, and based on its policy, selects an action based on the current state. It then receives feedback from the environment in the form of a reward and a new state.
2. In the second step, the critic uses the reward and the new state to calculate a **temporal difference** (**TD**) error. The TD error measures how far the critic's current estimate of the value function is from the observed outcome. The TD error is then the difference between the reward at time step *t* (plus a discount factor γ for the critic's estimates of the value of the next state *V(st+1)* to serve to balance the impact of immediate and future rewards) and the critic's estimates of the value of the current state *V(st)*.
3. The critic updates its value function parameters to minimize the TD error. This is done with gradient descent.
4. The actor is updated as well. The actor uses the TD error as a feedback signal. If the error is positive, it means that the action was better than expected and the actor should take it more often (increase the probability of taking this action in the future). If the error is negative, the actor should decrease the probability. The actor maximizes the policy using gradient ascent; we want to maximize the expected return.

Actor-critic methods work well with continuous action spaces, where value-based methods have problems. It is a stable and efficient method and reduces the variance of policy gradient updates. On the other hand, it is sensitive to hyperparameters, you have to train two networks, and it is more complex than Q-learning or REINFORCE.

Advantage Actor-Critic (A2C) is a popular variant where multiple agents interact with multiple instances of the environment in parallel. This allows for faster training.

AlphaZero

AlphaZero is a groundbreaking model-based RL algorithm developed by DeepMind in 2017, capable of mastering chess, shogi (Japanese chess), and Go. It has achieved superhuman performance, defeating human champions in these games. The success of AlphaZero lies in its innovative combination of deep learning and **Monte Carlo Tree Search (MCTS)**, which allows it to learn and plan effectively without human expertise or handcrafted rules.

AlphaZero learns entirely through **self-play**, starting with no prior knowledge other than the basic rules of the game. It plays millions of games against itself, gradually understanding what constitutes good or bad moves through trial and error. This self-play approach allows AlphaZero to discover optimal strategies, often surpassing even those developed by expert human players. Additionally, it enables the model to generate a vast amount of training data, far more than could be obtained by simply analyzing human games. The algorithm uses a deep neural network to represent both the policy (which actions to take) and the value function (the expected outcome of the game from a given state).

Traditional chess engines used to rely on game-tree search techniques. At each move, they would construct a game tree that represented all possible legal moves from the current position and performed a **depth-first search (DFS)** to a certain depth. This brute-force search examined all legal moves, assigning values to the final nodes based on heuristic evaluations formulated by the chess community. These heuristics, such as king safety, pawn structure, and control of the center, mimic factors used by human chess players to judge the quality of a move.

After evaluating the final nodes, traditional engines would backtrack and analyze the positions, pruning fewer promising branches to simplify the search. Despite these optimizations, this method had limitations, often leading to suboptimal moves and being computationally expensive. This is where MCTS comes in.

MCTS is an algorithm designed for decision-making in environments where planning several moves ahead is essential, especially in games with large state spaces where an exhaustive search is infeasible. MCTS builds a search tree by simulating games multiple times, gradually improving its understanding of the best actions through experience.

MCTS operates through four main steps, repeated to refine the search tree:

1. **Selection**: Starting from the root node (the current state), the algorithm selects child nodes using a strategy that balances exploration (trying less-explored moves) and exploitation (choosing moves that have shown promise). This is often done using the **Upper Confidence Bound for Trees (UCT)** formula, which considers both the average reward and the number of visits to each node.
2. **Expansion**: If the selected node is not a terminal state (the end of the game), the algorithm adds one or more child nodes, representing pos-

sible actions from this state. This expansion allows the search to cover new potential moves and outcomes.

3. **Simulation (rollout)**: From a newly added node, MCTS performs a simulation, or "rollout," by playing the game to a terminal state using a simple or random policy. The outcome of this simulation (win, loss, or draw) provides a reward, serving as an estimate of the value of the actions taken.
4. **Backpropagation**: The reward from the simulation is then backpropagated up the tree, updating the values associated with each node along the path to the root. This includes updating the average reward and the number of visits for each node. Over time, these updates help the algorithm determine which moves are most promising.

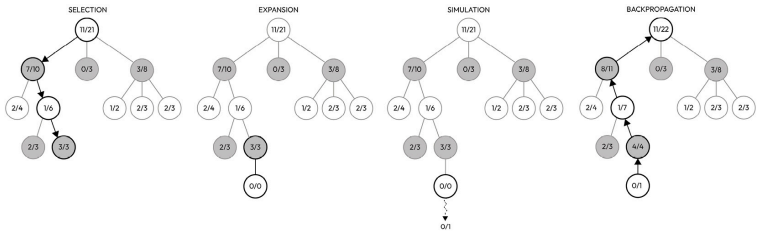


Figure 8.24 – Monte Carlo Tree Search (https://en.wikipedia.org/wiki/Monte_Carlo_tree_search)

AlphaZero then uses a neural network (a convolutional neural network that takes as input the arrangement of pieces on the board) and produces two outputs: policy head (a probability distribution over all possible moves, guiding the agent on which moves to consider) and value head (the likelihood of winning from the current board position, helping the agent to evaluate the strength of various states). AlphaZero uses MCTS to simulate potential moves and their outcomes (the agent plans several moves ahead in the game). Through MCTS, the model explores the moves that seem most promising and gradually improves its understanding of the game. The tree search uses the policy and value outputs from the neural network to prioritize which branches of the tree to explore. AlphaZero learns to play by playing against itself (self-play). In each game, the agent uses MCTS to decide moves and saves states (positions on the board), chosen moves, and results. This data is used to improve the policy and value estimates (neural network weight updates).

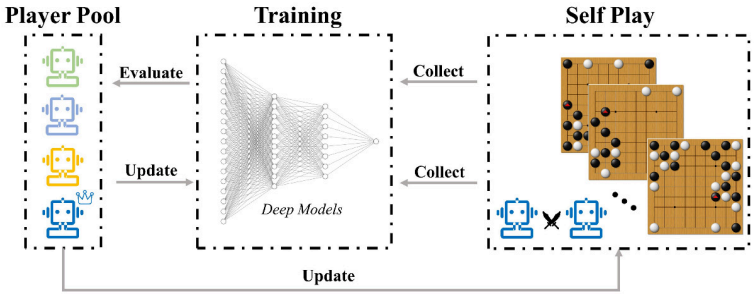


Figure 8.25 – AlphaZero pipeline (https://www.mdpi.com/2079-9292/10/13/1533)

AlphaZero therefore presents three main innovations:

- **Generalizing across games**: The same algorithm is used for three different games (chess, shogi, and Go), without the need for game-specific adjustments.
- **It requires no human knowledge**: Unlike traditional chess engines that use an extensive database of human games and strategies,

AlphaZero learns the game on its own. The model prioritizes strategies that offer long-term rewards within the game, rather than focusing solely on immediate benefits from individual moves. This approach enables the model to discover innovative strategies previously unexplored by humans or traditional chess engines.

- **Efficient search and learning**: Using MCTS and deep learning allows more efficient use of computational resources. Instead of conducting an extensive search of all possible moves, AlphaZero focuses only on the most promising moves.

Of course, AlphaZero is not without flaws either. The algorithm has a huge computational cost since it has to play millions of games against itself. Also, the algorithm works well for games (or settings where there is perfect information) but it is more difficult to adapt it to environments where the information is incomplete. Finally, there is discussion about actually understanding the game or learning abstract concepts, since the model fails some chess puzzles that are easy for humans.

In the next subsection, we will discuss the challenges with RL and new, exciting lines of research.

Challenges and future direction for deep RL

Although RL has made significant progress, several challenges and several active lines of research remain:

- **Generalization in unseen environments**: Generalization in environments that the agent has not seen remains a complex task. Agents are usually trained in a simulated environment or in specific settings, where they are able to excel after training. However, transferring learned skills to new environments, dynamic environments, or changing conditions is difficult. This limits the use of deep RL algorithms in the real world because real environments are rarely static or perfectly predictable. True generalization requires that a model not only learns solutions that are task-specific but also adapts to a range of situations (even if they did not occur during training).
- **Reward function design**: Reward function controls agent behavior, learning, and performance. Designing a reward function is difficult, especially in complex, scattered environments. In sparse reward settings, where there is limited feedback and it is often delayed, defining the reward and function is complex but critical. Even so, there is often a risk of creating bias, leading the policy to overfitting or unexpected behaviors, or making it suboptimal.
- **Compound error in model-based planning**: Model-based RL is at risk of compounding errors. The longer the horizon of predictions, the more errors accumulate in model predictions, leading to significant deviations from the optimal trajectory. This is especially the case for complex or high-dimensional space environments, thus limiting their use in real environments.
- **Multi-task learning**: Creating an agent that can be used for multiple tasks remains difficult, with the risk that the agent learns only the easier ones and ignores the more complex (or otherwise very poorly performing) ones. Also, a multi-task model often exhibits performance that is far inferior to an agent optimized for a single task. The design

of agents that can therefore be used for multi-task RL is difficult and still an active line of research.

- **Multi-modal RL**: With the advancement of computer vision and NLP, there are deep learning models that can either handle one mode individually or multiple modes together. This is why there is increasing discussion of using multimodal RL, where an agent can move through a multimodal environment and integrate information from the various modalities. For example, a robot can acquire information from the environment in an image and receive commands or instructions in natural language. An agent in video games receives visual information but also information from dialogues with characters or from other players. Multimodal learning remains complicated because an agent must simultaneously learn how to process multimodal information and optimize policy to interact in a complex environment. Similarly, it remains difficult to design a reward function for these cases.

In the next section, we will see how a neural network can be used to learn how to play a video game.

Learning how to play a video game with reinforcement learning

In this subsection, we will discuss how to train an agent to play a video game. In this case, the agent will be parameterized by a neural network. Following this policy, it will choose among the actions allowed by the video game, receive feedback from the environment, and use this feedback for parameter updating. In general, video games provide complex and dynamic environments that simulate real-world scenarios, thus making them an excellent testbed for RL algorithms. Video games provide a high-dimensional state space (pixel-based states, detailed universes) and a rich action space (discrete or continuous), are inspired by the real world, and can provide both immediate and delayed rewards (e.g., some actions may result in the direct death of the protagonist while a long-term strategy is needed to solve puzzles or win the game). In addition, many games require the user to explore the environment before they can master it. Enemies are often dynamic, and the model must learn how to defeat opposing agents or understand complex behaviors to overcome them. The game also provides clear rewards (which are often frequent or can otherwise be accelerated) that can then be easily defined for a reward function and thus make a safe playground (e.g., for algorithms for robotics). In addition, there are clear benchmarks and one can quickly compare the quality of a new algorithm.

We chose the actor-critic approach for this training because it has a number of features:

- Actor-critic can handle complex and continuous action spaces (control a character in a 3D environment) and thus can be used for a wide variety of games.
- The actor in the system learns the policy directly, making it efficient for scenarios where finding the policy is crucial. This is necessary in video games where quick decision-making and strategic planning are required.
- The critic provides feedback and speeds up learning in comparison to purely policy-based methods. Using a value function (critic) to evalu-

ate actions reduces the variance of policy updates, so it is more stable and efficient in environments where rewards are scattered.

- Actor-critic allows for efficient management of the balance between exploration and exploitation, where the actor explores the environment and the critic guides it by providing feedback. For more complex environments, actor-critic may not be sufficient, though it is a good initial choice and often sufficient.
- Actor-critic can also handle long-term planning. Often, in video games, there can be long-term rewards; the critic's value function helps the agent understand the long-term impact of its actions.
- Some variants are efficient in parallelizing and using data. A2C is a good choice for parallelizing environments and thus collecting more data, thus speeding up training and convergence.

We chose Super Mario as our game because it provides a rich and complex environment. The environment resembles the real world, and the representation of pixel-based observations as input is similar to that of real-world computer vision tasks, making Super Mario a good testbed for RL agents who need to learn to extract meaningful features from visual data. This environment is also partially observable, so it requires the agent to explore and learn about the environment. Different levels may require different strategies, so the model must be able to balance exploration and exploitation.

In the game, there are different kinds of challenges, such as navigating obstacles, facing different kinds of enemies, and learning to jump optimally and often dynamically. These different challenges represent different skills that an agent should develop: testing the agent's ability to make precise and timely actions (jumping over obstacles or gaps), assessing threats and deciding when to avoid or engage (avoiding or engaging enemies), and spatial awareness and strategic planning (navigating complex levels). The levels are progressive, so with a difficulty that progresses as the agent learns. In addition, there are both immediate rewards (collecting coins) and delayed rewards (e.g., completing a level), thus allowing for the evaluation of long-term strategies.

Finally, Super Mario has been widely adopted in the RL research community as a benchmark. Major libraries support it, or it is found directly integrated, thus allowing a quick way to test algorithms or conduct comparisons. There are also already well-researched strategies; the game is well documented and is a good example for both beginners and experts in RL. There are also implementations that are parallelizable, thus allowing effective and fast training.



Figure 8.26 – Super Mario screenshots from the training

All the code can be found within the repository, at the following link: **https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr8/RL_SuperMario**

Name	Last commit message	Last commit date
..		
tensorboard	Add Reinforcement learning Super Mario	19 hours ago
trained_models	Add Reinforcement learning Super Mario	19 hours ago
env.py	Add Reinforcement learning Super Mario	19 hours ago
evaluate_A3C.ipynb	Add Reinforcement learning Super Mario	19 hours ago
model.py	Add Reinforcement learning Super Mario	19 hours ago
optimizer.py	Add Reinforcement learning Super Mario	19 hours ago
process.py	Add Reinforcement learning Super Mario	19 hours ago
test.py	Add Reinforcement learning Super Mario	19 hours ago
train.py	Add Reinforcement learning Super Mario	19 hours ago
train_A3C.ipynb	Add Reinforcement learning Super Mario	19 hours ago

Figure 8.27 – Screenshot of the repository

Description of the scripts

To perform the training, we will use some popular RL libraries (OpenAI's Gym and PyTorch). In the repository, there are different scripts that are used to train the agent:

- **`env`**: This script defines the environment where our agent acts (Super Mario) and allows us to record a video of our agent playing, preprocess images for the model, define the reward function, set the world, set a virtual joystick, and more.
- **`model`**: This script defines a PyTorch neural network model for an actor-critic architecture. The model is designed to process image-like inputs, extract features, and then use those features to output both action probabilities (actor) and state value estimates (critic).
- **`optimizer`**: This code defines a custom optimizer class called **`GlobalAdam`**, which extends the functionality of PyTorch's built-in Adam optimizer.
- **`train`**: This script sets up and runs a distributed RL system using the **Asynchronous Advantage Actor-Critic** (**A3C**) method to train an agent to play Super Mario Bros.
- **`test`**: Model testing is in a separate script. This script allows you to load the trained model to play the game while rendering the gameplay.
- **`process`**: This script acts as the linking piece that integrates all the preceding components into a cohesive RL system for training and testing an agent to play Super Mario Bros.

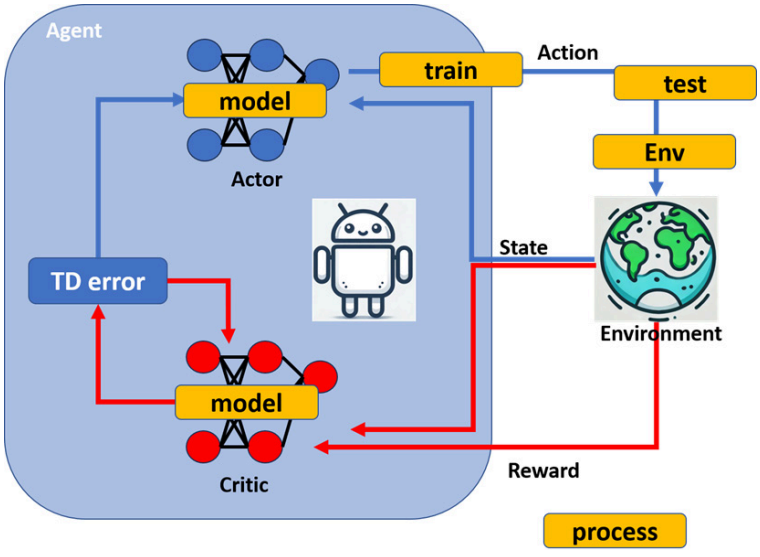


Figure 8.28 – Global view of the scripts

Setting up the environment

The **env** script allows us to have our setup of the environment, especially for RL algorithms such as Deep Q-Learning or actor-critic. Inside the script, we import the libraries we need, after which there are some functions that are used to create the world and define how the agent can interact with it:

- **Monitor**: The **Monitor** class allows the user to save a visual record of the agent's gameplay, which is useful for debugging, analyzing agent performance, and sharing results. This function permits us to save a video of the game using **.ffmpeg**.
- **process_frame**: The **process_frame** function is used to preprocess frames from the game to make them more suitable for training an RL agent. This function checks whether the frame is in the right format, converts it to grayscale and reduces the size, and normalizes it (simplifies the input). This allows the agent to focus on the important details of the visual information.
- **CustomReward**: This is a modification of the reward to encourage useful behaviors, track the current score, add rewards, check whether the agent finishes the level, and penalize it if the episode isn't finished. In this way, it tries to incentivize completing the level and making progress by penalizing failures.
- **CustomSkipFrame**: This serves to speed up training by allowing skip frames, thus reducing computational computation (fewer environment updates).
- **create_train_env**: This function sets up a fully customized and optimized Super Mario environment, making it ready for training an RL agent with efficient preprocessing, reward shaping, and frame skipping.

Defining the model

In the **model** script, we define the architecture for our algorithm. **ActorCritic** is the class that governs the architecture, and as a neural network, it is based on PyTorch (in fact, we use **nn.Module**, a classic neural network in PyTorch). The class has two components representing **Actor** (responsible for choosing actions) and **Critic**, which provides feedback. You can see that we have a shared feature extractor:

```
self.conv1 = nn.Conv2d(num_inputs, 32, 3, stride=2, padding=1)
self.conv2 = nn.Conv2d(32, 32, 3, stride=2, padding=1)
self.conv3 = nn.Conv2d(32, 32, 3, stride=2, padding=1)
self.conv4 = nn.Conv2d(32, 32, 3, stride=2, padding=1)
self.lstm = nn.LSTMCell(32 * 6 * 6, 512)
```

Here, we have a convolutional network to extract spatial features from the game; this output is then reshaped into a 2D tensor, which is passed for an LSTM. The LSTM has an update of the hidden state **hx** and the cell state **cx** (we described the LSTM in detail in ***Chapter 1***), thus managing episode memory.

After that, we initialize the two components:

```
self.critic_linear = nn.Linear(512, 1)
self.actor_linear = nn.Linear(512, num_actions)
```

Using a single feature extractor allows us to save computation resources. The two components produce two different outputs: **`actor_linear`** produces the output for the actor, which is a vector of size **`num_actions`**. This represents the probability of taking each action. The **`critic_linear`** component produces the output for the critic, which is a single scalar value. This value represents the estimated value of the current state (the expected return from this state). This separation allows us to make sure that the two layers have separate goals and different learning signals.

Next, we will define different loss functions in order to allow for different learning. As we can see, the two components produce different outputs:

```
def forward(self, x, hx, cx):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = F.relu(self.conv4(x))
        hx, cx = self.lstm(x.view(x.size(0), -1), (hx, cx))
        return self.actor_linear(hx), self.critic_linear(hx), hx, cx
```

Since we want our process to be optimized for distributed learning, we use a custom version of Adam. Adam is a classical optimizer that is used for updating the parameters of a neural network. The **`GlobalAdam`** class is designed for distributed RL, where multiple processes or agents share the same optimizer. The key idea is to make certain parts of the optimizer's state shared across processes, allowing agents to coordinate their updates to the model parameters efficiently. This is especially useful with actor-critic and especially the variant where there are many agents acting in the same environment. The idea is that we play the game several times asynchronously and then conduct global updates, reducing computation. The **`GlobalAdam`** script is simply an adaptation of Adam to RL problems, allowing averaging and learning from different processes:

```
import torch
class GlobalAdam(torch.optim.Adam):
    def __init__(self, params, lr):
        super(GlobalAdam, self).__init__(params, lr=lr)
        for group in self.param_groups:
            for p in group['params']:
                state = self.state[p]
                state['step'] = 0
                state['exp_avg'] = torch.zeros_like(p.data)
                state['exp_avg_sq'] = torch.zeros_like(p.data)
                state['exp_avg'].share_memory_()
                state['exp_avg_sq'].share_memory_()
```

Training the model

The **`train`** script then allows us to train the model asynchronously with different processes. The script allows us to provide several parameters (default parameters are already entered). For example, we can decide the level of the game (**`--world`** and **`--stage`**), the type of action (**`--action_type`**), the learning rate for the optimizer (**`--lr`**), hyperparameters specif-

ic to the algorithm and RL (**`--gamma`**, **`--tau`**, **`--beta`**), or related to the process and its parallelization (**`--num_processes`**, **`--num_local_steps`**, and **`--num_global_steps`**).

The **`train`** function allows us to initialize the training environment, initialize the policy, and use the GPU. The **`global_model.share_memory()`** method allows the global model's parameters to be accessible to all processes, enabling parallel updates. You can see we use **`GlobalAdam`** to update the global model's parameters. The **`torch.multiprocessing`** wrapper (is a wrapper to the multiprocessing module) allows us to create multiple processes that operate asynchronously. This script then defines the training of our model using multiple parallel processes. At the same time, the script allows easy configuration and customization:

```
def train(opt):
    torch.manual_seed(123)
    if os.path.isdir(opt.log_path):
        shutil.rmtree(opt.log_path)
    os.makedirs(opt.log_path)
    if not os.path.isdir(opt.saved_path):
        os.makedirs(opt.saved_path)
    mp = _mp.get_context("spawn")
    env, num_states, num_actions = create_train_env(opt.world, opt.stage, opt.actio
    global_model = ActorCritic(num_states, num_actions)
    if opt.use_gpu:
        global_model.cuda()
    global_model.share_memory()
    if opt.load_from_previous_stage:
        if opt.stage == 1:
            previous_world = opt.world - 1
            previous_stage = 4
        else:
            previous_world = opt.world
            previous_stage = opt.stage - 1
        file_ = "{}/A3CSuperMarioBros{}_{}".format(opt.saved_path, previous_world,
        if os.path.isfile(file_):
            global_model.load_state_dict(torch.load(file_))
    optimizer = GlobalAdam(global_model.parameters(), lr=opt.lr)
    processes = []
    for index in range(opt.num_processes):
        if index == 0:
            process = mp.Process(target=local_train, args=(index, opt, global_model
        else:
            process = mp.Process(target=local_train, args=(index, opt, global_model
        process.start()
        processes.append(process)
    process = mp.Process(target=local_test, args=(opt.num_processes, opt, global_mo
    process.start()
    processes.append(process)
    for process in processes:
        process.join()
```

Testing the system

The **`test`** script allows customization such as deciding on some parameters, such as level of play, actions, and so on. Once we have trained our model, we can then load it, play the game, and register the agent playing. The model then plays with its policy, without optimization in this script, and thus allows us to observe the agent's performance.

Connecting all the components

The **process** script connects all the scripts we have seen so far into one system. This script uses the **create_train_env** function from the **env** module to set up the Super Mario Bros game environment. This is the environment where our agent interacts and learns. The script also initializes the **ActorCritic** model (both actor and critic) and uses this model to make decisions and evaluate game states. The **local_train** function is responsible for training and requires the **GlobalAdam** optimizer. This script is also used to evaluate trained model performance, so it uses elements we defined in the test script. This script, then, is the central piece that allows us to have a fully functional RL system. It orchestrates the environment, model, and training algorithm, making everything work together to train an agent to play Super Mario Bros.

The **local_train** function enables the agent to train in parallel with other processes while updating a shared global model. This function establishes a seed for reproducibility, so we can reproduce the results. After that, we initialize the environment (**create_train_env**) and model (**ActorCritic**); if there is a GPU, we move the model to the GPU and initialize TensorBoard:

```
def local_train(index, opt, global_model, optimizer, save=False):
    torch.manual_seed(123 + index)
    if save:
        start_time = timeit.default_timer()
    writer = SummaryWriter(opt.log_path)
    env, num_states, num_actions = create_train_env(opt.world, opt.stage, opt.actio
    local_model = ActorCritic(num_states, num_actions)
    if opt.use_gpu:
        local_model.cuda()
```

At this point, we begin the training loop where each iteration represents an episode of gameplay. The local parameters are synchronized with the global parameters, and at the end of each episode, the hidden and cell states of the LSTM are reset:

```
local_model.train()
    state = torch.from_numpy(env.reset())
    if opt.use_gpu:
        state = state.cuda()
    done = True
    curr_step = 0
    curr_episode = 0
    while True:
        if save:
            if curr_episode % opt.save_interval == 0 and curr_episode > 0:
                torch.save(global_model.state_dict(),
                           "{}/a3c_super_mario_bros_{}_{}".format(opt.saved_path, o
            print("Process {}. Episode {}".format(index, curr_episode))
        curr_episode += 1
        local_model.load_state_dict(global_model.state_dict())
        if done:
            h_0 = torch.zeros((1, 512), dtype=torch.float)
            c_0 = torch.zeros((1, 512), dtype=torch.float)
        else:
            h_0 = h_0.detach()
            c_0 = c_0.detach()
        if opt.use_gpu:
```

```
            h_0 = h_0.cuda()
            c_0 = c_0.cuda()
```

At this point, we begin to collect experiences for a number of steps (**`opt.num_local_steps`**). Then, for a state, the model (the local model) produces a set of probabilities, and from these probabilities, we sample an action. Having chosen an action, we interact with the environment, so we get a reward and a new state. For each of these steps, we record the following: whether the episode has ended, the log probability of the action, the value estimate, the reward, and the entropy of the policy. If the episode ends, the state is reset, and the hidden states are detached to prevent gradient backpropagation:

```
for _ in range(opt.num_local_steps):
            curr_step += 1
            logits, value, h_0, c_0 = local_model(state, h_0, c_0)
            policy = F.softmax(logits, dim=1)
            log_policy = F.log_softmax(logits, dim=1)
            entropy = -(policy * log_policy).sum(1, keepdim=True)
            m = Categorical(policy)
            action = m.sample().item()
            state, reward, done, _ = env.step(action)
            state = torch.from_numpy(state)
            if opt.use_gpu:
                state = state.cuda()
            if curr_step > opt.num_global_steps:
                done = True
            if done:
                curr_step = 0
                state = torch.from_numpy(env.reset())
                if opt.use_gpu:
                    state = state.cuda()
            values.append(value)
            log_policies.append(log_policy[0, action])
            rewards.append(reward)
            entropies.append(entropy)
            if done:
                break
        R = torch.zeros((1, 1), dtype=torch.float)
        if opt.use_gpu:
            R = R.cuda()
        if not done:
            _, R, _, _ = local_model(state, h_0, c_0)
```

Now, it is time to calculate the loss and conduct backpropagation. Here, we use **generalized advantage estimation (GAE)**, to balance bias and variance and make the training therefore more efficient. Simply, the advantage function *A(s,a)* measures the goodness of an action *a* relative to the average action in a given state *s*. In the next script, GAE is used to compute the advantage values that drive the actor's policy updates. We use GAE to update the policy in the actor loss in order to maximize the expected return but keep the variance low. In other words, we want to keep the training more stable. By adding GAE, the training process becomes more efficient and less susceptible to noise from high variance returns or inaccuracies from biased value estimates:

```
        gae = torch.zeros((1, 1), dtype=torch.float)
        if opt.use_gpu:
            gae = gae.cuda()
```

```
        actor_loss = 0
        critic_loss = 0
        entropy_loss = 0
        next_value = R
        for value, log_policy, reward, entropy in list(zip(values, log_policies, re
            gae = gae * opt.gamma * opt.tau
            gae = gae + reward + opt.gamma * next_value.detach() - value.detach()
            next_value = value
            actor_loss = actor_loss + log_policy * gae
            R = R * opt.gamma + reward
            critic_loss = critic_loss + (R - value) ** 2 / 2
            entropy_loss = entropy_loss + entropy
        total_loss = -actor_loss + critic_loss - opt.beta * entropy_loss
        writer.add_scalar("Train_{}/Loss".format(index), total_loss, curr_episode)
        optimizer.zero_grad()
        total_loss.backward()
        for local_param, global_param in zip(local_model.parameters(), global_model
            if global_param.grad is not None:
                break
            global_param._grad = local_param.grad
        optimizer.step()
        if curr_episode == int(opt.num_global_steps / opt.num_local_steps):
            print("Training process {} terminated".format(index))
            if save:
                end_time = timeit.default_timer()
                print('The code runs for %.2f s ' % (end_time - start_time))
            return
```

Note that we have three separate losses. The first is the actor loss, which encourages actions that lead to higher rewards. The critic loss penalizes errors in the value estimation, and the entropy loss encourages exploration by penalizing overly confident action distributions (in other penalizing strategies that are too greedy). Once we have computed the total loss, we perform the backpropagation as in any neural network. Right now, we have performed local training, so we use the gradients of the local model to conduct the global model update as well. Every certain time interval, we save the model and send the loss logs to TensorBoard. The process ends when we have reached the total number of global steps.

The **`local_test`** function allows us to conduct the evaluation of our trained model. It runs as a separate process to test how well the agent performs using the learned policy:

```
def local_test(index, opt, global_model):
    torch.manual_seed(123 + index)
    env, num_states, num_actions = create_train_env(opt.world, opt.stage, opt.actio
    local_model = ActorCritic(num_states, num_actions)
    local_model.eval()
    state = torch.from_numpy(env.reset())
    done = True
    curr_step = 0
    actions = deque(maxlen=opt.max_actions)
    while True:
        curr_step += 1
        if done:
            local_model.load_state_dict(global_model.state_dict())
        with torch.no_grad():
            if done:
                h_0 = torch.zeros((1, 512), dtype=torch.float)
                c_0 = torch.zeros((1, 512), dtype=torch.float)
            else:
```

```
                h_0 = h_0.detach()
                c_0 = c_0.detach()
            logits, value, h_0, c_0 = local_model(state, h_0, c_0)
            policy = F.softmax(logits, dim=1)
            action = torch.argmax(policy).item()
            state, reward, done, _ = env.step(action)
            env.render()
            actions.append(action)
            if curr_step > opt.num_global_steps or actions.count(actions[0]) == actions
                done = True
            if done:
                curr_step = 0
                actions.clear()
                state = env.reset()
            state = torch.from_numpy(state)
```

Again, we conduct setup and initialization and load the local **`ActorCritic`** model in evaluation mode (in inference mode, practically, the model does not get updates during this process). At this point, we start the loop, where we load the last weights from the global model. For a state, we compute the probabilities for each action and choose the action with the highest probability. Note how, during training, we conducted sampling of the action; in evaluation mode, instead, we chose the action with a greedy policy. We interact with the environment and render the game, conduct action tracking, and check whether the agent gets stuck or repeats the same action indefinitely. If the agent exceeds the maximum number of steps or gets stuck, the episode ends and we reset the state. This function evaluates the performance of the trained agent, rendering the gameplay so that users can observe how well the agent has learned to play Super Mario Bros. It ensures the policy is effective and provides visual feedback.

Running the scripts, we can see that the training runs in parallel:

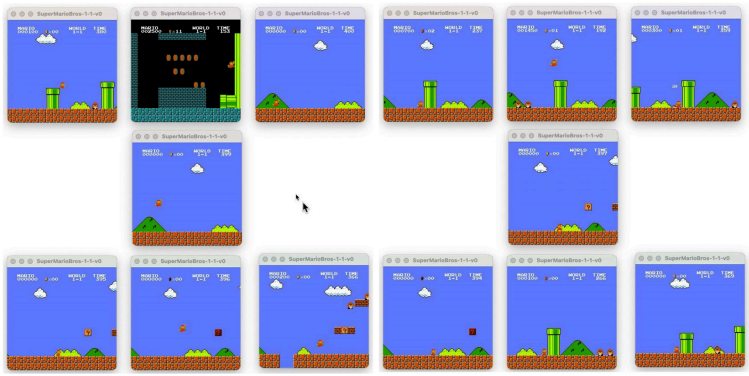


Figure 8.29 – Screenshot of the script run

You can check out the video here: **https://www.youtube.com/watch?v=YWx-hnvqjr8**

To summarize, we used several scripts to implement a variant of the action-critic algorithm (the A3C method). This method involves training multiple agents in parallel to explore the environment, collect experiences, and update a shared global model asynchronously. In other words, we use a variant that allows us to speed up the training and learn a model that is more robust because it retrieves different experiences from different agents. For cleaner organization, we divided the process into several scripts that are then linked into a single script (process script). We defined our neural network with a common extractor for the two compo-

nents, so we can save some computations. In addition, we used an LSTM to be able to handle the temporal dependencies there are between one state and another. We had to modify our optimizer because we needed shared memory to be able to handle several processes to update a global model. Asynchronous training indeed has higher complexities, where each agent needs to access and update the global model. After that, we defined how to train our model by collecting some experience. Having collected the experience, we conducted an update of the model weights, calculating the loss and performing backpropagation. Periodically, we synchronized the global and local model, conducting an update of the global model. After that, we defined how to evaluate our agent using the parameters of the global model. The agent uses the learned policy to play the game.

These scripts allow efficient parallel training because of the A3C method. In fact, we can use several agents in parallel that explore the environment, gather experience, and then lead to a global model update. Using a parallel system causes agents to explore different parts of the environment, leading to more diverse experiences and thus a more generalized policy. In general, this is favorable because different strategies may be needed in video games. In the same vein, we added entropy loss to encourage exploration and prevent the agent from being stuck in a suboptimal strategy. The script is designed for efficient use of resources, to reduce computation, and to have fast training (we did not add an experience replay buffer to save memory and thus consume less RAM). The use of a global model ensures that knowledge learned by one agent is immediately available to all agents; this usually promotes rapid convergence.

The choice of an on-policy learning method such as A3C can result in high variance in policy updates. This variance can then be amplified by the asynchronous nature, which may make it difficult to get consistent results across runs. In fact, the asynchronous approach introduces non-determinism, meaning that the results can vary significantly between runs. This makes the process less predictable and complicates the choice of hyperparameters (which is why we have provided default parameters, although it is possible to experiment with them). While we have tried to optimize the resources consumed by this script, the whole process remains resource intensive (like RL in general).

A3C primarily relies on CPU-based parallelism; however, incorporating GPU-friendly methods could significantly enhance training efficiency. Algorithms such as PPO can leverage GPUs to optimize the training process. Effective use of GPUs enables more efficient batch processing, allowing for the accumulation of experiences and bulk updates to the model. For readers interested in exploring GPU-based optimization, here are a few potential ideas:

- Test different hyperparameters and vary their values to better understand their impact. In the script, you can easily set and change hyperparameters. We invite you to test lambda (λ) to find a better balance between bias and variance.
- Try PPO. PPO is a popular alternative to A3C that exploits multiple epochs of mini-batch updates. As we have seen, it is an algorithm that promotes stability and works well in many cases. It also does not require many hyperparameters and the default ones usually work well.

- Adopt synchronous A2C as it is a simpler, synchronous version of A3C. This approach collects experiences in parallel and uses batches for updating. It is usually slower but easier to debug.

The model shown in this project can be applied to several other video games, showing how an RL algorithm can solve real tasks.

LLM interactions with RL models

RL algorithms have been instrumental for agents that can navigate complex environments, optimize strategies, and make decisions, with successes in areas such as robotics and video games. LLMs, on the other hand, have had a strong impact on **natural language processing** (**NLP**), enabling machines to understand human language and instructions. Although potential synergies can be imagined, so far these two technologies have evolved in parallel. In recent years, though, with the heightened interest in LLMs, the two fields have increasingly intersected. In this section, we will discuss the interaction between RL and LLMs.

We can have three cases of interaction:

- **RL enhancing an LLM**: Using RL to enhance the performance of an LLM in one or more NLP tasks
- **LLMs enhancing RL**: Using LLMs to train an RL algorithm that performs a task that is not necessarily NLP
- **RL and LLMs**: Combining RL models and LLMs to plan a skill set, without either system being used to train or conduct fine-tuning of the other

Let's discuss these in detail.

RL-enhanced LLMs

We have already discussed alignment and prompt engineering, in ***Chapter 3***. RL is, then, used for fine-tuning, prompt engineering, and the alignment of LLMs. As mentioned in ***Chapter 3***, LLMs are trained to predict the next word in a sequence, leading to a mismatch between the LLM's training objective and human values. This can lead LLMs to produce text with bias or other unsafe content, and likewise to be suboptimal at following instructions. Alignment serves to realign the model to human values or to make an LLM more effective for safer deployment. One of the most widely used techniques is **reinforcement learning from human feedback** (**RLHF**), where the reward is inferred from human preferences and then used to train the LLM. This process follows three steps: collect human feedback data, train a reward model on this data, and conduct fine-tuning of the LLM with RL. Generally, the most popular choice of RL algorithm is PPO or derivative methods. In fact, we do not want our aligned model to be significantly different from the original model, which PPO guarantees.

Interaction with LLMs is through the prompt, and the prompt should condense all the instructions for the task we want LLM to accomplish. Some work has focused on using RL to design prompts. Prompt optimization can be represented as an RL problem with the goal of incorporating human knowledge and thus drawing interpretable and adaptable prompts. The agent is used to construct prompts that are query-dependent and op-

timized. One can also train a policy network to generate desired prompts, with the advantage that the prompts are generally transferable across LLMs. An intriguing aspect of this approach is that some of these optimized prompts are grammatically "gibberish," indicating that high-quality prompts for a task need not follow human language patterns.

LLM-enhanced RL

LLM-enhanced RL refers to methods that use multi-modal information processing, generation, reasoning, or other high-level cognitive capabilities of pre-trained LLMs in assisting an RL agent. In other words, the difference from traditional RL is the use of an LLM and the exploitation of its knowledge and capabilities in some way. The addition of an LLM in some form has a twofold advantage: first, an LLM possesses reasoning and planning skills that allow for improved learning, and second, it has a greater ability to generalize. In addition, an LLM has extensive knowledge gained during the pre-training step and that can be transferred across domains and tasks, thus allowing better adaptation to environments that have not been seen. Models that are pre-trained generally cannot expand their knowledge or acquire new capabilities (continual learning is an open challenge of deep learning), so using a model trained with huge amounts of knowledge can help with this aspect (LLMs are generalists and have huge amounts of information for different domains in memory).

An LLM can then be inserted into the classic framework of an RL system (an agent interacting with and receiving feedback from an environment) at more than one point. An LLM can then be integrated to extract information, reprocess state, redesign rewards, make decisions, select actions, interpret policies, analyze world similarity, and more.

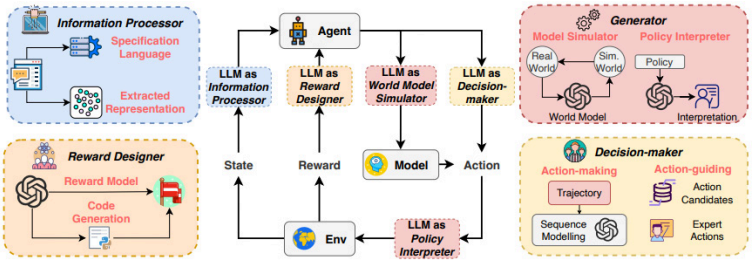


Figure 8.30 – Framework of LLM-enhanced RL in classical agent-environment interactions (https://arxiv.org/pdf/2404.00282)

Thus, an LLM can be used inside the system as an information processor, reward designer, decision-maker, and generator.

Information processor

When a task requires textual information or visual features, it can be complex for an agent to understand the information and optimize the policy at the same time. As we saw earlier, a convolutional neural network can be used to process images for a model to interact with a video game or board game. In the case of a chatbot, we can then use a model that understands language. Alternatively, instead of using a model directly on the language, we can use an LLM to extract features that allow the agent to learn more quickly. LLMs can be good feature extractors, thereby reducing the dimensionality and complexity of the information. Or LLMs can translate natural language into a specific formal language under-

standable to an agent. For example, in the case of a robot, the natural language of different users will be different and not homogeneous, making it difficult for the agent to learn. An LLM can transform instructions into a standard, formal language that allows easier learning for the agent.

A wide pre-trained model learns a representation of the data that can then be used for subsequent applications. A model, then, can be used to extract a data representation that we can use to train an agent. An LLM can be used frozen (that is, without the need for further training) to extract a compressed representation of the history of the environment. Some studies use an LLM to summarize past visual observations that are provided to the agent, so we can provide a memory to the agent. Using a frozen model is clearly the simplest alternative, but when agents are deployed in the real world, performance can degrade rapidly due to real-world variations versus the training environment. Therefore, we can conduct fine-tuning of both the agent and the LLM. The use of a feature extractor (an LLM or other large model) makes it easier for the agent to learn since these features are more invariant to changes in the environment (changes in brightness, color, etc...), but on the other hand, they have an additional computational cost.

The capabilities of LLMs can be used to make the task clearer. For example, instructions in natural language can be adapted by an LLM into a set of instructions that are clearer to the agent (for example, when playing a video game, a textual description of the task could be transformed into a set of instructions on how to move the character). An LLM can also be used to translate an agent's surroundings into usable information. These approaches are particularly promising but are currently limited in scope.

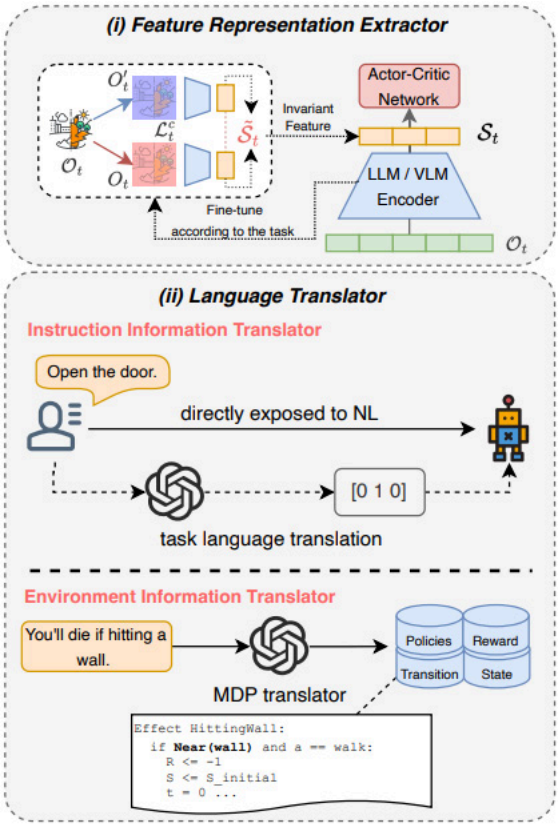


Figure 8.31 – LLM as an information processor (https://arxiv.org/pdf/2404.00282)

Reward designer

When knowledge of the problem is available, or when the reward can be defined by a clear and deterministic function (such as a game score or a win/loss condition), designing the reward function is straightforward. For example, in Atari games (or other games), it is easy to draw a reward function (e.g., victory represents a positive signal and defeat a negative signal). There are many applications where this is not possible because the tasks are long and complex, the rewards are scattered, and so on. In such cases, the knowledge inherent in an LLM (the knowledge gained during pre-training, coding abilities, and reasoning skills) could be used to generate the reward. It can be used indirectly (an implicit reward model) or directly (an explicit reward model). For example, a user can define expected behavior in a prompt, and an LLM can evaluate the agent's behavior during training, providing a reward and a penalty. So, you can use direct feedback from the LLM, or an LLM can generate the code for a reward function. In the second approach, the function can be modified by the LLM during training (for example, after the agent has acquired some skills, making it harder to get a reward).

An LLM can be an implicit reward model that provides a reward (or auxiliary reward) based on the task description. One technique for this is direct prompting, in which instructions are given to the LLM to evaluate the agent's behavior or decide on a reward. These approaches can mimic human feedback to evaluate an agent's behavior in real time. Alternatively, an alignment score can be used, for example, between the outcome of an action and the goal (in other words, evaluating the similarity between the expected outcome and reality). In some approaches, one uses the contrastive alignment between language instructions and the image observations of the agent, thus exploiting models that are multimodal. Obviously, the process of aligning human intentions and LLM-reward generation is not easy. There can be ambiguities, and the system does not always work with low-quality instruction, but it seems a promising avenue.

An explicit reward model exploits the ability of an LLM to generate code, thus generating a function (making the decision-making and reward-generation process by the LLM more transparent). This allows functions for subgoals to be generated automatically (e.g., having a robot learn low-level tasks using high-level instructions that are translated into a reward function by the LLM). The main limitation of this approach is the common-sense reasoning limitation of LLMs. LLMs are not capable of real reasoning or true generalization, so they are limited by what they have seen during pre-training. Highly specialized tasks are not seen by LLMs during pre-training, thus limiting the applicability of these approaches to a selected set of tasks. Adding context and additional information could mitigate this problem.

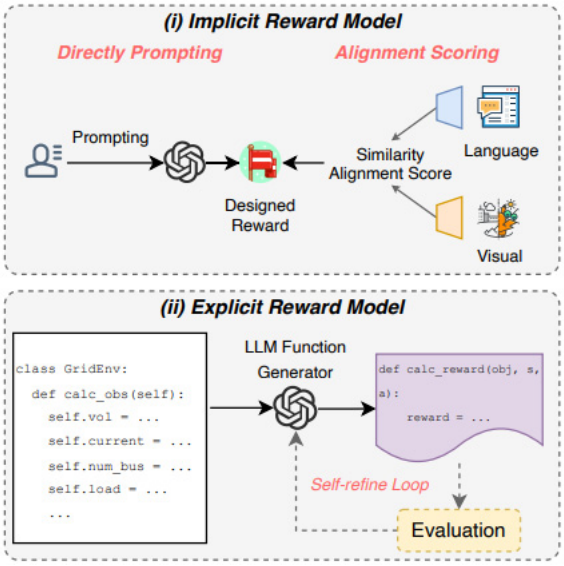


Figure 8.32 – LLM as a reward designer (https://arxiv.org/pdf/2404.00282)

Decision-maker

Since RL has problems in many cases with sample and exploration inefficiency, LLMs can be used in decision-making and thus help in choosing actions. LLMs can be used to reduce the set of actions in a certain state (for example, when many actions are possible). Reducing the set of actions reduces the exploration space, thus increasing exploration efficiency. For example, an LLM can be used to train robots on what actions to take in a world, reducing exploration time.

The transformer (or derivative models) has shown great potential in RL. The idea behind it is to treat these problems as sequence modeling problems (instead of trial and error). LLMs can then be seen as a decision-making model, which has to decide on a sequence of problems (as we mentioned in ***Chapter 2***, the transformer is trained on a sequence of problems, so making a decision on a sequence of states is congenial to its training). An LLM can be fine-tuned to leverage the internal representation of the model. In fact, in this way, we leverage the representation learned from an LLM (being trained with a huge quantity of text, an LLM has a vast amount of knowledge already acquired that can be applied to a task) to decide an action. Using prior knowledge reduces the need for data collection and exploration (hence, sample efficiency) and makes the system more efficient toward long-term rewards or sparse reward environments. Several studies have shown not only the transferability of knowledge learned from an LLM to other models but also the improved performance of the whole system on different benchmarks. In addition, vision-language models can be used to be able to adapt the system to multimodal environments. Using an LLM as a decision-maker is still computationally expensive (even if only used in inference and without the need for fine-tuning). As a result, current studies are focusing on trying to reduce the computational cost of these approaches.

Alternatively, an LLM can guide the agent in choosing actions by generating reasonable action candidates or expert actions. For example, in environments such as text-based games, the action space is very large, and only a fraction of the actions is currently available, so an agent can learn with extensive trial-and-error; however, this exploration is very inefficient. An LLM can reduce this action space by generating an action set by

understanding the task. This makes it possible to reduce exploration and make it more efficient, collect more rewards, and speed up training. Typically, in these approaches, we have an LLM that generates a set of actions and another neural network that generates the Q-values of these candidates. The same approach has been extended to robots that have to follow human instructions, where an LLM generates possible actions. This approach is limited owing to the inheritance of the biases and limitations of an LLM (since an LLM decides the action space and generates it according to its knowledge and biases).

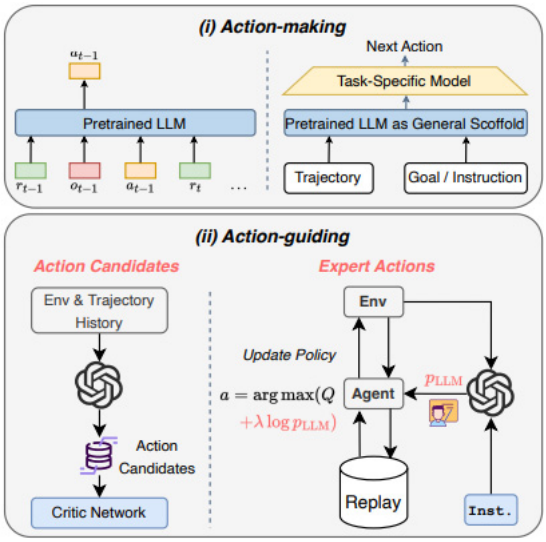


Figure 8.33 – LLM as a decision-maker (https://arxiv.org/pdf/2404.00282)

Generator

Model-based RL relies on world models to learn the dynamics of the environment and simulate trajectories. The capabilities of an LLM can be to generate accurate trajectories or to explain policy choices.

An LLM has an inherent generative capacity that allows it to be used as a generator. An LLM can then be used as a world model simulator, where the system generates accurate trajectories that the agent uses to learn and plan. This has been used with video games, where an LLM can generate the trajectories and thus reduce the time it takes an agent to learn the game (better sample efficiency). The LLM's generative capabilities can then be used to predict the future. Although promising, there is still difficulty in aligning the abstract knowledge of an LLM with the reality of an environment, limiting the impact of its generative capability.

Another interesting approach is where an LLM is used to explain the policy of an RL system. **Explainable RL (XRL)** is a subfield at the intersection of explainable machine learning and RL that is growing. XRL seeks to explain an agent's behavior clearly to a human being. An LLM could then be used to explain in natural language why an agent makes a certain decision or responds in a certain way to a change in environment. As a policy interpreter, an LLM given a state and an action should explain an agent's behavior. These explanations should then be understandable to a human, thus allowing an agent's safety to be checked. Of course, the quality of the explanations depends on the LLM's ability to understand the representation of the features of the environment and the implicit logic of the policy. It is difficult to use domain knowledge or examples to improve understanding of a complex policy (especially for complex environments).

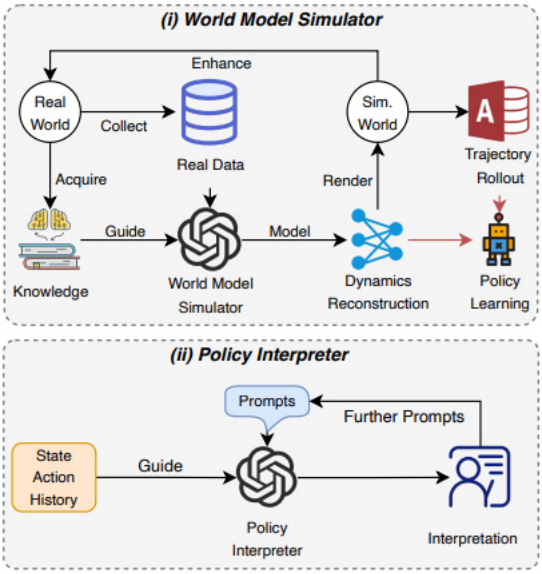


Figure 8.34 – LLM as a generator (https://arxiv.org/pdf/2404.00282)

LLM-enhanced RL can be useful in a variety of applications:

- **Robotics**: Using LLMs can improve the interaction between humans and robots, help robots better understand human needs or human logic, and improve their decision-making and planning capabilities.
- **Autonomous driving**: RL is used in autonomous driving to make decisions in changing environments that are complex and where input from different sensors (visual, lidar, radar) must be analyzed along with contextual information (traffic laws, human behavior, unexpected problems). LLMs can improve the ability to process and integrate this multimodal information, better understand instructions, and improve the goal and rewards (e.g., design reward functions that take into account not only safety but also passenger comfort and engine efficiency).
- **Healthcare recommendations**: RL is used in healthcare to learn recommendations and suggestions. LLMs can be used for their vast knowledge and ability to analyze huge amounts of patient data and medical data, accelerating the agent's learning process, or providing information for better learning.
- **Energy management**: RL is used to improve the use, transportation, conversion, and storage of energy. In addition, it is expected to play an important role in future technologies such as nuclear fusion. LLMs can be used to improve sample efficiency, multitask optimization, and much more.

Despite all these opportunities, there are also a number of limitations to the use of LLMs in RL. The first challenge is that the LLM-enhanced RL paradigm is highly dependent on the capabilities of the LLM. LLMs suffer from bias and can hallucinate; an agent then inherits these problems from the LLM. In addition, LLMs can also misinterpret the task and data, especially when they are complex or noisy. In addition, if the task or environment is not represented in their pre-training, LLMs have problems adapting to new environments and tasks. To limit these effects, the use of synthetic data, fine-tuning the model, or use of continual learning has been proposed. Continual learning could allow a model to adapt to new tasks and new environments, without forgetting what the model has learned previously. To date, though, continual learning and catastrophic forgetting are open problems in deep learning.

In addition, the addition of an LLM brings a higher computational cost (both in training and in inference), and an increase in latency time. Several techniques can be used to reduce this computational cost, such as quantization, pruning, or using small models. Some approaches use *mixture of experts*, allowing conditional computation, transformer variants (state space models), caching strategies, and so on.

Finally, one should not forget that the use of LLMs also opens up ethical, legal, and safety issues. The same problems we saw in ***Chapter 3*** are also applicable to these systems. For example, data privacy and intellectual property remain open topics for applications in sensitive fields such as healthcare or finance.

Key takeaways

Because this chapter was dense in terms of theory, we decided to add a small recap section. This chapter introduced RL as a core approach to enabling intelligent agents to learn from interaction with dynamic environments through trial and error, similar to how humans learn by acting, observing outcomes, and adjusting behavior. RL differs from supervised learning by focusing on learning from rewards rather than labeled data, and it is especially suited to tasks with delayed feedback and evolving decision sequences.

RL is a machine learning paradigm where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards. It learns through trial and error, balancing exploration (trying new actions) and exploitation (using known strategies).

In summary, we have these classes of methods:

- **Model-free versus model-based RL**:
  - **Model-free methods** (e.g., DQN, REINFORCE) learn directly from interaction without modeling the environment. They are simpler and more scalable.
  - **Model-based methods** use an internal model to simulate outcomes and plan ahead. They are more sample-efficient and suitable for environments where planning is crucial but are harder to design and compute.
- **On-policy versus off-policy methods**:
  - **On-policy methods** learn from the data generated by the current policy (e.g., REINFORCE, PPO), making them more stable but sample inefficient.
  - **Off-policy methods** (e.g., DQN) can learn from past or alternative policies, improving sample efficiency and exploration flexibility.
- **Main algorithms discussed**:
  - **Q-Learning and DQN**: Learn value functions using lookup tables or neural networks.
  - **REINFORCE**: A basic policy gradient method using stochastic policies.
  - **PPO**: Balances stability and performance by clipping policy updates.
  - **Actor-Critic**: Combines value estimation and policy learning for more robust updates.
  - **AlphaZero**: Combines deep learning with Monte Carlo Tree Search for self-play-based strategy optimization in complex games.

- **Practical use cases**:
  - **Gaming**: RL agents such as AlphaZero and DQN have mastered games such as Go, Chess, and Atari titles.
  - **Robotics**: RL allows robots to learn complex movement and interaction policies through simulation and real-world feedback.
  - **Autonomous vehicles**: RL enables the learning of driving strategies in dynamic and uncertain environments.
  - **Optimization and control**: Applied in finance, healthcare, logistics, and industrial automation for sequential decision-making.

Summary

In the previous chapters, the main question was how to find information and how to deliver it effectively to an LLM. In such cases, the model is a passive agent that receives information and responds. With this chapter, we are trying to move away from this paradigm, toward an idea where an agent explores an environment, learns through this exploration, performs actions, and learns from the feedback that the environment provides to it. In this view, the model is an active component that interacts with the environment and can modify it. This view is also much closer to how we humans learn. In our exploration of the external world, we receive feedback that guides us in our learning. Although much of the world has been noted in texts, the real world cannot be reduced to a textual description. Therefore, an agent cannot learn certain knowledge and skills without interacting with the world. RL is a field of artificial intelligence that focuses on an agent's interactions with the environment and how it can learn from it.

In this chapter, therefore, we introduced the fundamentals of RL. In the first section, we discussed the basic components of an RL system (agent, environment, reward, and action). We then discussed the main question of RL, how to balance exploration and exploitation. Indeed, an agent has a goal (accomplish a task) but learns how to accomplish this task through exploration. For example, we saw in the multi-armed bandit example how a greedy model performs worse than a model that explores the possibilities. This principle remains fundamental when we define an agent to solve complex problems such as solving a video game. To solve complex tasks, we introduced the use of neural networks (deep RL). We saw that there are different types of algorithms with different advantages and disadvantages, and we saw how we can set one of them to win in a classic video game. Once we trained our model, we discussed how LLM and RL fields are increasingly intersecting. In this way, we saw how the strengths of the two fields can be synergistic.

From this chapter on, the focus will be more applicative. We will see how an agent can generally accomplish a task. In the upcoming chapters, the agent will mainly be an LLM who will use tools to perform actions and accomplish tasks. The choice, then, for the agent will not be which action to take but which tool to choose in order to accomplish a task. Despite the fact that an LLM agent interacts with the environment, one main difference is that there will be no training. Training an LLM is a complex task, so in these systems, we try to train them as little as possible. If, in the previous chapters (*5–7*), we tried to leverage the comprehension skills of an LLM, in the next chapters, we will try to leverage the skills of LLMs to in-

teract with the environment or with other agents – skills that are possible anyway because an LLM can understand a task and instructions.

Further reading

- Ghasemi, *An Introduction to Reinforcement Learning: Fundamental Concepts and Practical Applications*, 2024, https://arxiv.org/abs/2408.07712
- Mnih, *Playing Atari with Deep Reinforcement Learning*, 2013, https://arxiv.org/abs/1312.5602
- Hugging Face, *Proximal Policy Optimization (PPO)*, https://hugging-face.co/blog/deep-rl-ppo
- Wang, *Learning Reinforcement Learning by LearningREINFORCE*, https://www.cs.toronto.edu/~tingwuwang/REINFORCE.pdf
- Kaufmann, *A Survey of Reinforcement Learning from Human Feedback*, 2024, https://arxiv.org/pdf/2312.14925
- Bongratz, *How to Choose a Reinforcement-Learning Algorithm*, 2024, https://arxiv.org/abs/2407.20917v1
- Schulman, *Proximal Policy Optimization Algorithms*, 2017, https://arx-iv.org/abs/1707.06347
- OpenAI, *Proximal Policy Optimization*, https://openai.com/index/ope-nai-baselines-ppo/
- OpenAI Spinning UP, *Proximal Policy Optimization*, https://spin-ningup.openai.com/en/latest/algorithms/ppo.html
- Bick, *Towards Delivering a Coherent Self-Contained Explanation of Proximal Policy Optimization*, 2021, https://fse.studentthe-ses.ub.rug.nl/25709/1/mAI_2021_BickD.pdf
- Silver, *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, 2017, https://arxiv.org/abs/1712.01815
- McGrath, *Acquisition of Chess Knowledge in AlphaZero*, 2021, https://arxiv.org/abs/2111.09259
- DeepMind, *AlphaZero: Shedding New Light on Chess, Shogi, and Go*, 2018, https://deepmind.google/discover/blog/alphazero-shedding-new-light-on-chess-shogi-and-go/
- Gao, *Efficiently Mastering the Game of NoGo with Deep Reinforcement Learning Supported by Domain Knowledge*, 2021, https://www.mdpi.-com/2079-9292/10/13/1533
- Francois-Lavet, *An Introduction to Deep Reinforcement Learning*, 2018, https://arxiv.org/abs/1811.12560
- Tang, *Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes*, 2024, https://arxiv.org/abs/2408.03539
- Mohan, *Structure in Deep Reinforcement Learning: A Survey and Open Problems*, 2023, https://arxiv.org/abs/2306.16021
- Cao, *Survey on Large Language Model-Enhanced Reinforcement Learning: Concept, Taxonomy, and Methods*, 2024, https://arxiv.org/abs/2404.00282

Table of contents collapsed